

CUADERNO DE DIVULGACIÓN · DEEP REINFORCEMENT  
LEARNING

# Arkanoid DRL Learning Lab

Cuatro algoritmos de aprendizaje por refuerzo aprendiendo a jugar al Arkanoid en el navegador, con redes neuronales **reales** sobre TensorFlow.js y aceleración **WebGPU**. Sin trucos: el agente no conoce las reglas del juego; las descubre jugando.

DQN

PPO

SAC

World Model

TensorFlow.js

WebGPU

AUTOR

Roberto Canales Mora

LICENCIA

MIT

REPOSITORIO

[github.com/rcanalescoder/DRLArkanoid](https://github.com/rcanalescoder/DRLArkanoid)

---

Un laboratorio para entender el RL profundo, de dentro afuera.

Roberto Canales Mora · con Claude Chat / Code

# Aprender jugando, como tú

La idea esencial del aprendizaje por refuerzo.

Olvida por un momento la palabra «inteligencia artificial». Imagina que le enseñas a alguien un juego que no ha visto nunca, pero **sin explicarle las reglas**. Lo único que puedes decirle, después de cada movimiento, es «**bien**» o «**mal**». Al principio se moverá al azar y fallará; pero si insiste, irá notando qué movimientos le dan más «bien» y los repetirá. Eso es, en esencia, el **aprendizaje por refuerzo**, y es lo que ocurre en este Arkanoid.



## La analogía del adiestramiento

Cuando enseñas un truco a un perro no le explicas anatomía ni física: le das un premio cuando lo hace bien. Con muchas repeticiones, el perro asocia «esta acción → premio» y la repite. El agente hace lo mismo, pero el premio es un número.

## Las cuatro palabras clave



### El agente

El «jugador». No es una persona: es la red neuronal que mira la pantalla y decide cómo mover la pala. Es quien aprende.



### La acción

Lo único que el agente puede hacer aquí: mover la pala a la **izquierda**, a la **derecha** o **dejarla quieta**. Tres opciones, nada más.



### La recompensa

El «bien/mal» convertido en número: **+1** por romper un ladrillo, **-1** por perder la bola. El agente solo quiere sumar lo máximo posible.



### El episodio

Una partida entera, desde el saque hasta perder la bola (o ganar). El agente juega **miles** de episodios; en cada uno mejora un poquito.

## Los primeros minutos del agente

Un agente recién creado no sabe nada. Esta es la evolución típica conforme entrena, y explica la forma de la curva de recompensa:

| MOMENTO                    | QUÉ HACE EL AGENTE                                                    | LA RECOMPENSA...                         |
|----------------------------|-----------------------------------------------------------------------|------------------------------------------|
| <b>Segundo 0</b>           | Mueve la pala al azar. La bola cae al vacío casi siempre.             | muy baja (pierde la bola enseguida).     |
| <b>Primer minuto</b>       | Empieza a intuir que conviene poner la pala <b>debajo</b> de la bola. | sube: ya devuelve la bola algunas veces. |
| <b>Después</b>             | Devuelve la bola con soltura y va rompiendo ladrillos.                | claramente positiva y al alza.           |
| <b>Con suerte y tiempo</b> | Aprende a colar la bola arriba y reventar varios de golpe.            | se dispara (combos).                     |

Que la recompensa sea **negativa al principio** es esperable: el agente aún está aprendiendo lo más básico, evitar perder la bola. Empieza a subir en cuanto aprende a devolverla.

### En esencia

Un agente que **prueba**, recibe **premios y castigos** y **repite lo que funciona**. El resto son las herramientas técnicas para hacerlo bien y a gran escala.

# De un ladrillo a una política

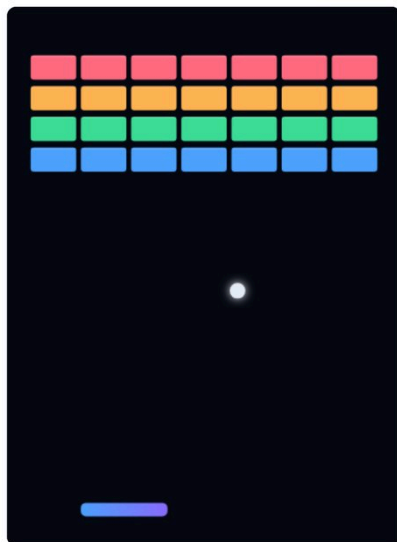
Qué es esto, quiénes son los protagonistas y cómo está pensado para aprenderse.

El **aprendizaje por refuerzo** (RL, *Reinforcement Learning*) es la rama de la IA en la que un **agente** aprende a base de **premios y castigos**: prueba acciones, observa la recompensa y ajusta su comportamiento para acumular el máximo premio a largo plazo. Aquí lo vemos sobre un Arkanoid simplificado, donde el agente solo puede mover una pala a izquierda/derecha o dejarla quieta, y debe aprender a devolver la pelota y romper ladrillos **sin que nadie le explique las reglas**.

Así se ve el laboratorio completo en marcha: arriba, el juego y las decisiones del agente en tiempo real; debajo, las curvas y paneles que cuentan si está aprendiendo. A lo largo de estas páginas iremos abriendo cada una de estas piezas.

**Motivación del laboratorio.** Esta interfaz está pensada como guía de estudio: no solo muestra al agente jugando, también el **mecanismo interno**. Por eso hay un icono de información (i) junto a cada grupo y cada dato importante. **Pulsa cualquier** (i) para abrir una ficha completa que explica qué es, por qué importa y, cuando aplica, su fórmula y sus datos.

PARTIDA OBSERVADA · ENV 0 · jugando



Recompensa episodio: **-0.03** · Ladrillos: 0/28 · Pasos: 4

MÉTRICAS CLAVE

RECOMPENSA-100  
**1.295**  
▲ 0.000

PÉRDIDA  
**0.000**

E EXPLORACIÓN  
**0.050**

TD-ERROR  
**0.000**

BUFFER  
**38.400**

TASA DE ÉXITO  
**0.0%**

Inspector DQN

Q(s,a) — valor esperado de cada acción

← 0.000

· 0.000

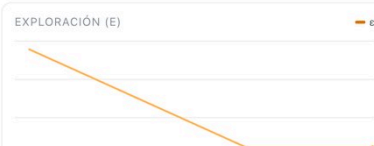
→ 0.000

ε exploración **0.050**

TD-error medio **0.0000**

Buffer **38.400 (38%)**

CURVAS DE ENTRENAMIENTO



TRANSICIÓN ACTUAL · s (0.59, 0.53) → a ← → r -0.008 → s' (0.57, 0.51) → done false

VERSIÓN DEL EXPERIMENTO

DQN

Model-free · basado en valor · off-policy

recomendado

PPO

Model-free · actor-crítico · on-policy

política

SAC

Model-free · actor-crítico · off-policy

avanzado

World Model

Model-based · off-policy

model-based

CONTROL DE EJECUCIÓN

Velocidad de simulación 4x

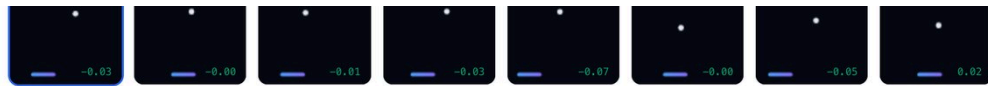
Entornos de entrenamiento (headless) 256

Entornos visuales 8

☒ Reward shaping potencial

ENTORNOS EN PARALELO





#### CONCEPTOS DE DRL — DQN

|                                                                               |         |
|-------------------------------------------------------------------------------|---------|
| Model-free — Aprende sin modelar el entorno                                   | ✓ Usado |
| Model-based — Aprende un modelo del entorno y planifica                       | no usa  |
| Replay buffer — Memoria de experiencias pasadas                               | ✓ Usado |
| Red objetivo — Copia estable para calcular objetivos                          | ✓ Usado |
| Exploración $\epsilon$ -greedy — Acción aleatoria con probabilidad $\epsilon$ | ✓ Usado |
| Política estocástica — Distribución de probabilidad sobre acciones            | no usa  |
| Ventaja (GAE) — ¿Cuánto mejor fue una acción que la media?                    | no usa  |
| Objetivo recortado — Limita el cambio de política por paso                    | no usa  |
| On-policy / rollouts — Entrena solo con datos de la política actual           | no usa  |
| Doble crítico (clipped double-Q) — Dos redes Q, se usa la menor               | no usa  |
| Máxima entropía — Recompensa también ser impredecible                         | no usa  |
| Temperatura $\alpha$ automática — Ajusta el peso de la entropía solo          | no usa  |
| Modelo de dinámica — Red que predice $(s,a) \rightarrow (s',r, done)$         | no usa  |
| Planning / imaginación — Entrena con experiencia simulada por el modelo       | no usa  |

#### FLUJO DE DATOS



Off-policy: las experiencias se guardan en un replay buffer y se reutilizan en minibatches aleatorios, rompiendo la correlación temporal y mejorando la eficiencia de datos.

#### REPLAY BUFFER (CONCEPTUAL) — experiencias y su recompensa

Cada experiencia es una transición (estado, acción, recompensa, siguiente). Las más informativas—una bola perdida o un ladrillo roto— tienen mayor TD-error (más "sorpresa" para la red) y, con replay prioritario, se muestrean con más frecuencia. replay prioritario disponible

|  |                  |                                  |             |
|--|------------------|----------------------------------|-------------|
|  | (s, a, +1, s')   | Ladrillo roto · TD-error 0.82    | <div></div> |
|  | (s, a, +0.2, s') | Rebote correcto · TD-error 0.34  | <div></div> |
|  | (s, a, -1, done) | Bola perdida · TD-error 0.93     | <div></div> |
|  | (s, a, +5, done) | Nivel completado · TD-error 0.76 | <div></div> |

#### LAS CUATRO FAMILIAS EN ESTE LABORATORIO

MODEL-FREE · BASADO EN VALOR · OFF-POLICY  
**DQN** — Deep Q-Network  
Aprende la función de valor-acción  $Q(s,a)$  y actúa de forma  $\epsilon$ -greedy. Usa replay buffer y red objetivo. Aquí: Double DQN + pérdida Huber + soft update.

MODEL-FREE · ACTOR-CRÍTICO · ON-POLICY  
**PPO** — Proximal Policy Optimization  
Optimiza directamente la política con un objetivo recortado (clipped surrogate) y una crítica de valor. On-policy: recoge rollouts y los procesa en varias épocas. Usa GAE para estimar ventajas.

MODEL-FREE · ACTOR-CRÍTICO · OFF-POLICY  
**SAC** — Soft Actor-Critic (discreto)  
Actor-crítico de máxima entropía. Dos críticos Q (reduce sobreestimación), política estocástica y temperatura  $\alpha$  ajustada automáticamente para mantener la exploración. Versión discreta (Christodoulou 2019).

MODEL-BASED · OFF-POLICY  
**World Model** — World Model · Dyna-Q  
Aprende un modelo de la dinámica del entorno  $(s,a) \rightarrow (s',r)$  y lo usa para generar experiencias imaginadas con las que entrenar la política (Dyna-Q). Combina datos reales e imaginados.

El laboratorio al completo (una sola pantalla, partida en dos para que quepa): juego e información de aprendizaje.

## ¿En qué se diferencia de otras IAs?

Hay tres grandes formas de que una máquina aprenda. En el **aprendizaje supervisado** le damos ejemplos con la respuesta correcta (“esta foto es un gato”). En el **no supervisado** le damos datos sin etiquetar y busca patrones. El **aprendizaje por refuerzo** es distinto: **nadie le dice la respuesta correcta**; el agente **actúa**, recibe una recompensa y descubre por sí mismo qué conviene hacer. Aprende por **ensayo y error**, como un ser vivo.

### Aprendizaje supervisado

datos con la  
respuesta correcta

“esto es un gato”

### Aprendizaje no supervisado

datos sin etiqueta;  
busca estructura

“estos se parecen”

### Aprendizaje por refuerzo (RL)

premios y castigos  
al actuar

“+1 si ganas”

El RL no aprende de respuestas correctas, sino de la recompensa que recibe al probar.

Las tres familias del aprendizaje automático. El RL aprende interactuando y recibiendo premios/castigos, no de respuestas dadas.



### Una analogía: aprender a montar en bici

Nadie te da “la fórmula” del equilibrio. Pruebas, te tambaleas (castigo), aciertas un tramo (premio) y tu cuerpo ajusta solo lo que funciona. Eso es exactamente el RL: probar, medir el resultado y reforzar lo que sale bien.

## Los protagonistas, en una frase

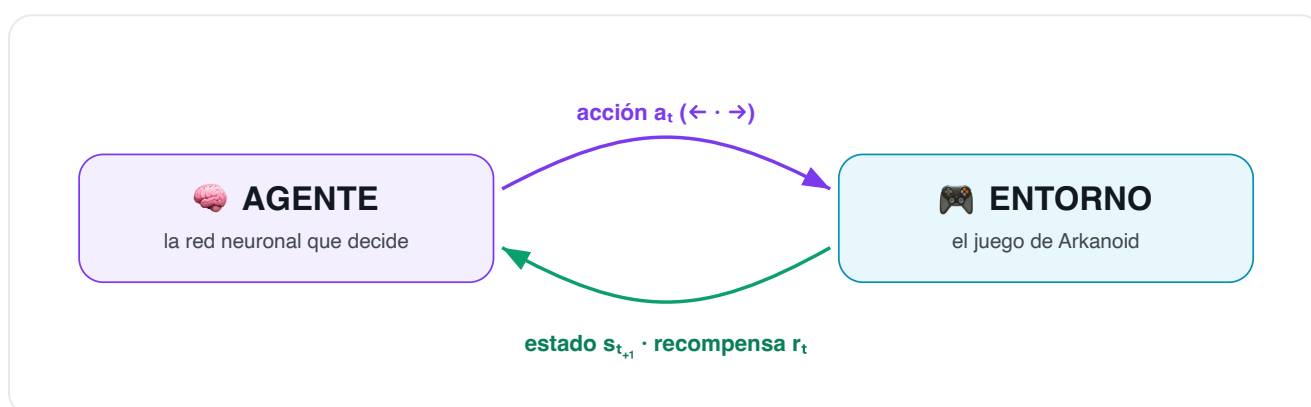
|                   |                                                                                                                |
|-------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Entorno</b>    | El juego de Arkanoid: recibe una acción, avanza la física un paso y devuelve una recompensa.                   |
| <b>Estado</b>     | Lo que el agente “ve” en cada instante: 6 números (posición y velocidad de la pelota, posición de la pala...). |
| <b>Acción</b>     | Una de 3: mover la pala a la izquierda, mantener, o a la derecha.                                              |
| <b>Recompensa</b> | El número que premia (romper un ladrillo: +1) o castiga (perder la pelota: -1) cada paso.                      |
| <b>Agente</b>     | La red (o redes) neuronal que decide qué acción tomar y que aprende de la experiencia.                         |
| <b>Política</b>   | La estrategia del agente: la función que, dado un estado, dice qué acción conviene.                            |

# El vocabulario esencial

Los conceptos en los que se apoyan los cuatro algoritmos.

## El bucle principal: agente ↔ entorno

El RL modela una conversación que se repite millones de veces entre dos partes. En cada **paso de tiempo**  $t$ : el **agente** observa el estado  $s_t$ , elige una **acción**  $a_t$ ; el **entorno** aplica esa acción, evoluciona y devuelve la **recompensa**  $r_t$  y el siguiente estado  $s_{t+1}$ . El agente usa esa información para mejorar y vuelve a empezar. El objetivo del agente es maximizar la **recompensa acumulada** a largo plazo, no solo la inmediata.



El bucle de interacción. El agente actúa, el entorno responde con estado y recompensa, y el ciclo se repite.

## Los componentes de un sistema de RL

### Estado (s)

Lo que el agente percibe en un instante. Aquí, 6 números: posición y velocidad de la pelota, posición de la pala y la distancia pelota-pala. Es la "foto" sobre la que decide.

### Acción (a)

Lo que el agente puede hacer. Aquí, 3 acciones discretas: mover la pala a la izquierda, mantenerla o moverla a la derecha.

### Recompensa (r)

La señal numérica que premia o castiga cada paso. Define qué significa "jugar bien". El agente no busca otra cosa que acumular recompensa.

### Política ( $\pi$ )

La estrategia del agente: una función que, dado un estado, decide la acción (o una probabilidad por acción). Es lo que de verdad queremos aprender.

### Valor (V / Q)

Cuánta recompensa futura cabe esperar. **V(s)** mide lo bueno que es un estado; **Q(s,a)**, lo bueno que es tomar una acción en un estado. Guían a la política.

### Episodio

Una partida completa. Termina de tres formas: perder la pelota, limpiar el nivel, o agotar el límite de pasos (600 = "tiempo agotado", sin castigo). El agente aprende de muchos episodios; medimos la recompensa media de los últimos 100.

### Exploración vs explotación

El dilema central: ¿pruebo algo nuevo (explorar) por si es mejor, o aprovecho lo que ya sé que funciona (explotar)? Demasiada explotación estanca; demasiada exploración no avanza.

### Memoria (replay)

Un almacén de experiencias pasadas (s,a,r,s') del que se muestrean lotes para entrenar. Rompe la correlación temporal y reutiliza cada dato. Propio de métodos off-policy.

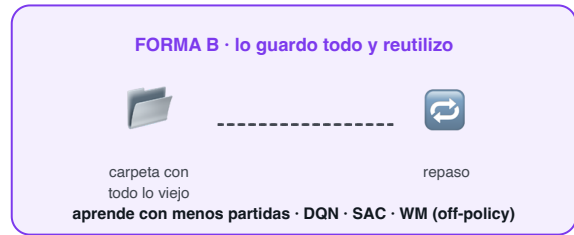
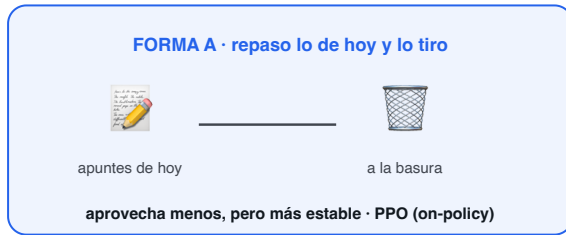
### Modelo del entorno

Una predicción de cómo responde el entorno: dado (s,a), ¿cuál es s' y r? La mayoría de métodos no lo usan (model-free); el World Model sí (model-based).

## ¿Aprender solo de lo último, o también de lo viejo?

Hay dos «escuelas», y se entienden pensando en cómo estudiarías para un examen. **Forma A:** repasas solo lo de hoy y por la noche tiras los apuntes; mañana, apuntes nuevos. **Forma B:** guardas todos los apuntes viejos en una carpeta y repasas un poco de todo, también lo de hace semanas.

## ¿Aprovecha solo lo último o también lo viejo? Dos formas de estudiar



Dos formas de aprovechar la experiencia: tirar los «apuntes» de cada tanda de partidas, o guardarlos y reutilizarlos.

El agente aprende igual. En la **Forma A** solo aprende de su **última tanda de partidas** y la olvida enseguida: aprovecha menos cada partida, pero es más estable (así es PPO). En la **Forma B guarda partidas viejas en una memoria** y las reutiliza muchas veces: aprende con menos partidas, pero a veces se vuelve más inestable (así son DQN, SAC y World Model).

**On-policy y off-policy.** La Forma A se denomina **on-policy**: el método solo aprende de los datos de su política actual. La Forma B se denomina **off-policy**: puede aprender también de experiencias anteriores, guardadas en una memoria llamada **replay buffer**.

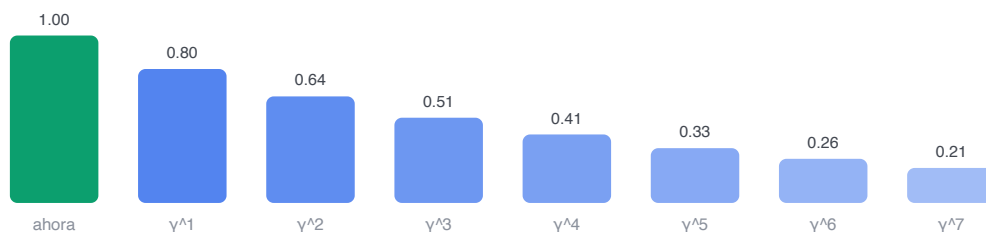
## Recompensa, retorno y descuento

Cuidado con una confusión habitual: el agente **no** maximiza la recompensa de *un* paso, sino el **retorno** (G): la suma de **todas** las recompensas futuras. Pero el futuro lejano es incierto, así que se **descuenta** con un factor  $\gamma$  (gamma) entre 0 y 1: cada paso que se aleja en el tiempo cuenta un poco menos.

### PARA CURIOSOS: LA FÓRMULA DEL RETORNO

$G = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 + \gamma^3 \cdot r_3 + \dots$  Con  $\gamma=0.99$  el agente es muy previsor (el futuro casi no se descuenta); con  $\gamma \rightarrow 0$  sería cortoplacista (solo le importaría el premio inmediato).

El retorno  $G = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 + \dots$  ( $\gamma$  = factor de descuento, 0.99 aquí)



Las recompensas futuras valen menos: el agente prefiere el premio pronto, pero no ignora el futuro.

El descuento: las recompensas futuras pesan menos ( $\gamma^t$ ). Por eso el agente prefiere ganar pronto, pero sin ignorar lo que vendrá.

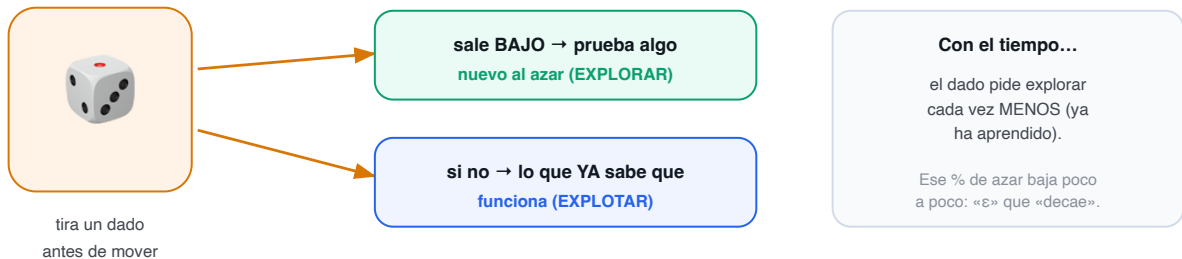
### Mini-ejemplo: calcular un retorno

1. Imagina que en los próximos 3 pasos ganas  $r = 1, 0, 1$  y que  $\gamma = 0.9$ .
2. El retorno es  $G = 1 + 0.9 \cdot 0 + 0.9^2 \cdot 1 = 1 + 0 + 0.81 = 1.81$ .
3. Fíjate: ese último «+1», por estar dos pasos en el futuro, solo cuenta como 0.81. Cuanto más lejos, menos pesa. Con  $\gamma=0.99$  (lo que usamos) pesaría casi igual: el agente es muy

## Probar cosas nuevas o ir a lo seguro

El agente tiene un dilema constante: ¿prueba un movimiento nuevo (por si es mejor) o repite lo que ya sabe que funciona? La forma más simple de decidirlo es **echarlo a suertes**. Imagina que, antes de cada movimiento, tira un dado: si sale un número bajo, prueba una acción **al azar**; si no, hace **lo que ya conoce**.

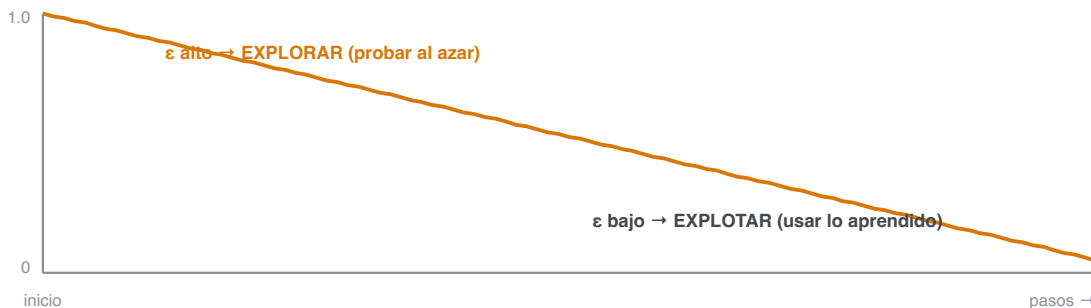
¿Probar algo nuevo o ir a lo seguro? El agente lo echa a suertes



La receta más simple para decidir entre probar (explorar) y ir a lo seguro (explotar): un poco de azar antes de cada movimiento.

Al principio le interesa probar mucho (todavía no sabe nada), así que el dado manda explorar muy a menudo. Según aprende, prueba cada vez menos: ese porcentaje de azar **va bajando poco a poco**.

**ε-greedy: la probabilidad de actuar al azar baja con el tiempo**

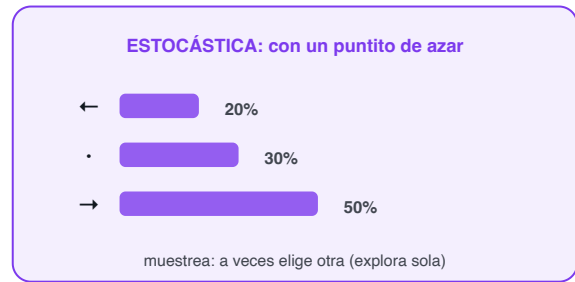
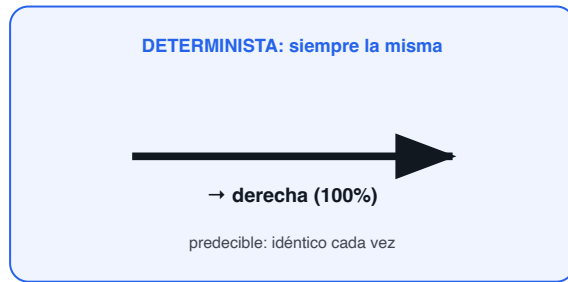


El porcentaje de movimientos «al azar» empieza alto (probar mucho) y baja con el tiempo hacia un mínimo (ya casi solo aprovecha lo aprendido).

**Épsilon (ε) y ε-greedy.** El porcentaje de movimientos al azar se denomina **épsilon (ε)**, y la estrategia de explorar con esa probabilidad e ir a lo seguro el resto del tiempo, **ε-greedy** (la emplean DQN y el World Model). Que ε **decae** significa que disminuye con el entrenamiento.

PPO y SAC exploran de otra manera: su política ya incorpora **cierta aleatoriedad** al elegir, de modo que no siempre toma la misma acción. La diferencia entre elegir siempre lo mismo (predecible) y repartir entre varias acciones:

### Dos formas de que el «cerebro» elija acción

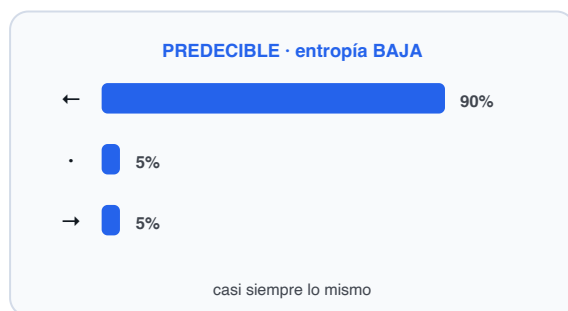


Elegir siempre lo mismo (a la izquierda) frente a elegir con algo de azar repartido entre las acciones (a la derecha).

**Política estocástica.** Una política que elige la acción con cierta aleatoriedad (no siempre la misma) se denomina **estocástica**. La que elige siempre la misma es **determinista**.

SAC, además, **premia mantener variedad** en sus decisiones: un agente que siempre hace lo mismo deja de descubrir alternativas mejores. Esa variedad también se mide con un número:

**Entropía = cuánta VARIEDAD (sorpresa) hay en sus decisiones**



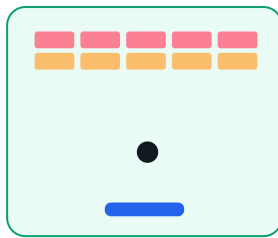
Cuando casi siempre elige lo mismo, hay poca «variedad» (entropía baja); cuando reparte entre varias acciones, hay mucha (entropía alta).

**Entropía.** La medida de la variedad de las decisiones de la política se denomina **entropía**. Más entropía equivale a más exploración; SAC la premia para no concentrarse en pocas acciones demasiado pronto.

## ¿Cómo de buena es una situación? El «valor»

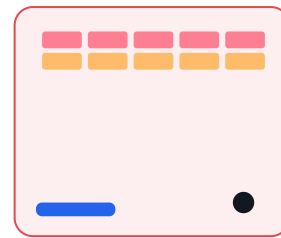
Cuando juegas, intuyes al vuelo si una jugada «pinta bien» o si «vas a perder». El agente aprende a ponerle un **número** a esa intuición: cuántos puntos espera conseguir a partir de esa situación. Si la pala está justo bajo la bola y quedan ladrillos, pinta bien (número alto). Si la bola se escapa por un lado, pinta mal (número bajo).

El «valor» de una situación = ¿cómo de prometedora pinta?



BIEN: la pala está bajo la bola  
valor alto ( $\approx +4$ )

La red aprende  
a ponerle un  
número a cada  
situación.



MAL: la bola se escapa por el lado  
valor bajo ( $\approx -1$ )

El «valor» es el número que la red le pone a cada situación: cuánto premio espera conseguir desde ahí en adelante.

**Valor,  $V(s)$  y  $Q(s, a)$ .** Ese número se denomina **valor**.  $V(s)$  indica cómo de buena es una situación;  $Q(s, a)$ , cómo de buena es tomar una acción concreta en esa situación. Estimar bien estos números es buena parte del problema.

## El truco para no mirar infinitos pasos (Bellman)

¿Cómo estimamos el valor de algo si depende de todo el futuro? El truco de **Bellman** es **recursivo**: el valor de tomar una acción ahora = la recompensa inmediata **más** el valor (descontado) de lo mejor que pueda hacer después. Así, en vez de mirar infinitos pasos, solo miramos **un paso y el valor del siguiente estado**. Casi todo el RL basado en valor sale de aquí.

La idea de Bellman: descomponer el problema en 'recompensa de ahora + lo mejor que venga después'



$$Q(s, a) = r + \gamma \cdot \max Q(s', a') \leftarrow \text{el valor de ahora se define con el valor del futuro}$$

La ecuación de Bellman para Q: el valor de  $(s, a)$  se define con la recompensa  $r$  y el mejor valor del estado siguiente  $s'$ .

### Ejemplo paso a paso: calcular un objetivo

1. El agente rompe un ladrillo: recompensa  $r = +1$ , el episodio no termina ( $\text{done} = 0$ ).
2. La red objetivo estima que, desde el estado siguiente, la mejor acción vale  $Q'(s', a^*) = 3.0$ . Usamos  $\gamma = 0.99$ .
3. Objetivo =  $r + \gamma \cdot (1 - \text{done}) \cdot Q'(s', a^*) = 1 + 0.99 \cdot 1 \cdot 3.0 = 3.97$ .
4. Si la red creía que  $Q(s, a) = 2.5$ , el **TD-error** es  $3.97 - 2.5 = 1.47$ : una sorpresa grande  $\rightarrow$  experiencia muy informativa (¡por eso el replay prioritario la repetiría más!).

### Comprueba que lo entiendes

- ¿Por qué  $Q(s, a)$  depende de  $Q(s', \cdot)$  y no solo de  $r$ ?

- Si  $\text{done} = 1$  (el episodio termina), ¿cuánto aporta el término del futuro?
- ¿Qué significa, en una frase, un TD-error grande?

## Las recompensas de este Arkanoid

Una recompensa de solo  $+1$  /  $-1$  sería **pobre**: el agente recibiría señal en muy pocos momentos. Aquí usamos varios eventos de distinta magnitud para dar pistas más ricas y frecuentes. Esta es la **lista completa** de recompensas del entorno (todas se suman en cada paso para formar la recompensa del paso):

| EVENTO                      | RECOMPENSA                               | PARA QUÉ                                                                                                                                                        |
|-----------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cada paso (vivir)           | 0.0                                      | No penalizamos sobrevivir: el agente no tiene prisa por terminar. (Podría ponerse un pequeño coste negativo para incentivar la rapidez.)                        |
| Romper un ladrillo          | +1.0                                     | El objetivo del juego. Premio claro y repetible.                                                                                                                |
| Combo (ladrillos seguidos)  | $+0.5 \cdot (n-1)$                       | Bonus por cada ladrillo extra de una misma subida, sin que la bola vuelva a la pala. El 1.º suma +1, el 2.º +1.5, el 3.º +2... Premia reventar varios de golpe. |
| Devolver la pelota (rebote) | +0.2                                     | Premio intermedio: enseña a no perder la pelota antes de saber romper ladrillos.                                                                                |
| Perder la pelota            | -1.0                                     | Castigo terminal: termina el episodio. Le enseña a colocar la pala.                                                                                             |
| Completar el nivel          | +5.0                                     | Gran premio terminal por limpiar los 28 ladrillos.                                                                                                              |
| Acercar la pala a la pelota | shaping $\approx$<br>$+0.3 \cdot \Delta$ | Recompensa densa y telescópica (potential shaping): da señal en cada paso sin cambiar la política óptima. Acelera mucho el aprendizaje inicial.                 |

### Diseñar recompensas para enseñar una estrategia

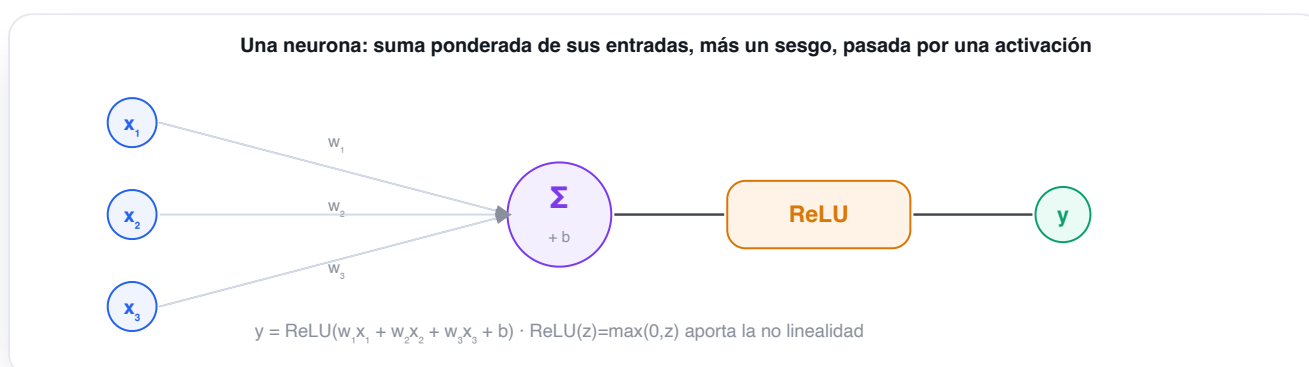
Fíjate en el **combo**: con él no solo premiamos romper ladrillos, sino romper **muchos de una**. Así empujamos al agente hacia la jugada maestra del Breakout —colar la bola por un lateral para que rebote arriba y reviente una hilera entera— **sin programársela**: la **descubre** porque le da más recompensa. Ese es el arte del **reward shaping**: elegir bien los premios para que la conducta deseada emerja sola.



Panel "Replay buffer (conceptual)" de la app: las experiencias y su recompensa real. Las más informativas (perder la bola, romper un ladrillo) tienen mayor TD-error y se repiten más con replay prioritario.

## De la neurona a la red

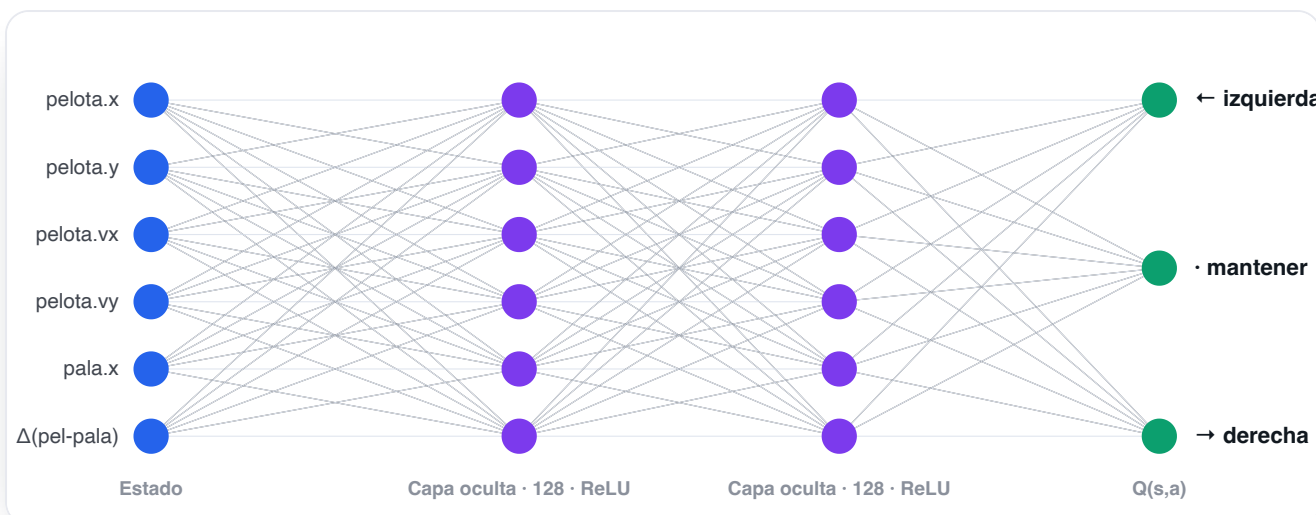
La pieza básica es la **neurona**: toma varias entradas, las combina con unos **pesos** (su importancia), suma un **sesgo** y pasa el resultado por una **activación** (aquí **ReLU**, que deja pasar lo positivo y pone a cero lo negativo). Apilando muchas neuronas en **capas** obtenemos una red capaz de aprender relaciones complejas.



Una neurona: suma ponderada de las entradas ( $w \cdot x$ ) más un sesgo ( $b$ ), pasada por una activación no lineal (ReLU).

## La red neuronal que usamos

¿Cómo "aprende" el agente? Mediante una **red neuronal**: una función con miles de parámetros (pesos) que se ajustan para acertar cada vez más. La nuestra es un **perceptrón multicapa (MLP)**: toma los 6 números del estado, los pasa por **dos capas ocultas de 128 neuronas** con activación **ReLU** (que introduce la no linealidad necesaria para aprender relaciones complejas) y produce la salida. Según el algoritmo, esa salida es distinta: en **DQN** y **World Model** son los 3 valores **Q(s,a)**; en **PPO/SAC**, el actor da 3 probabilidades (política) y un crítico da 1 valor.



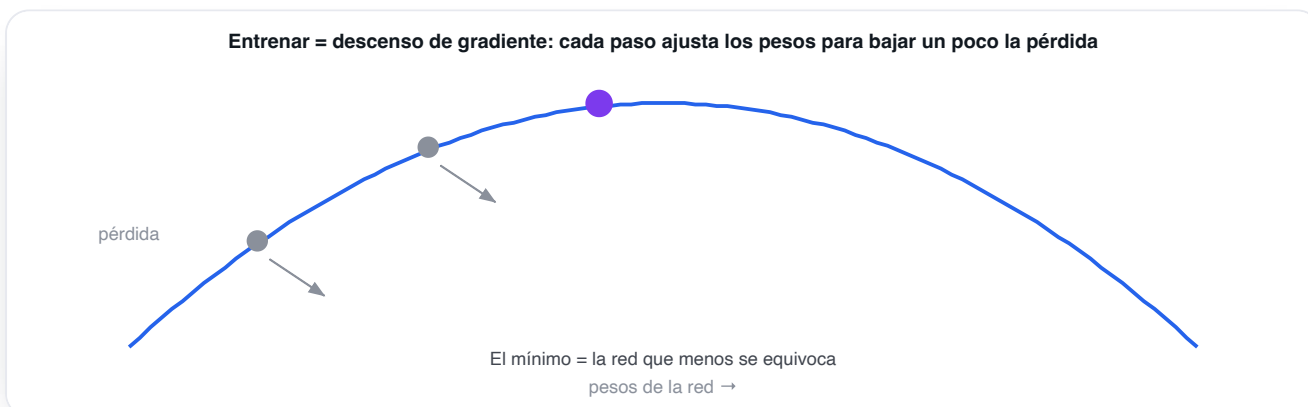
La red de este laboratorio: 6 entradas (el estado)  $\rightarrow$  128  $\rightarrow$  128 (ReLU)  $\rightarrow$  3 salidas (un valor o probabilidad por acción).  
Cada línea es un peso que el entrenamiento ajusta.

### Aproximar una función

Con pocos estados podríamos guardar  $Q$  en una tabla. Pero el estado aquí es continuo (infinitas posiciones de la pelota): imposible tabular. La red **aproxima** esa función y, además, **generaliza** a estados que nunca vio exactamente. Eso es lo "deep" del Deep RL.

## Aprender = ir afinando los mandos

¿Y cómo «aprende» de verdad la red? Imagina que afinas una radio antigua girando el dial **poco a poco** hasta que se oye claro, o un termostato hasta dar con la temperatura justa. La red hace lo mismo con sus miles de mandos internos (sus **pesos**): mide cuánto se ha equivocado y mueve cada mando **un poquito** en la dirección que reduce el error. Repetido millones de veces, va «sonando» cada vez mejor.



Cada ajuste mueve los mandos «cuesta abajo», hacia donde la red se equivoca menos. El fondo del valle es la versión de la red que mejor acierta.

**Pérdida, descenso de gradiente y learning rate.** A «cuánto se equivoca» se le llama **pérdida**; al método de reducirla ajustando los pesos, **descenso de gradiente**; y al tamaño de cada ajuste, **learning rate** (tasa de aprendizaje). Pasos grandes: rápido pero inestable; pasos pequeños: lento pero estable.

## Red actual y red objetivo

Para entrenar necesitamos un **objetivo** al que acercar las predicciones. En RL ese objetivo se construye con la propia red (ecuación de Bellman), así que si usáramos la misma red para predecir y para fijar su

objetivo, este se movería a cada paso: como intentar alcanzar tu propia sombra. La solución son **dos copias** de la red:

#### ► Red actual (online)

La que **entrenamos** en cada paso y la que **decide** las acciones. Cambia continuamente.

#### 🎯 Red objetivo

Una **copia que se mueve despacio** y solo sirve para calcular el objetivo. Da un blanco estable al que apuntar.



#### PARA CURIOSOS: CÓMO SE «MUEVE DESPACIO» LA COPIA

En vez de copiar la red entera de golpe, la objetivo copia **un poquito** de la actual en cada paso:  $\theta^- \leftarrow \tau \cdot \theta + (1-\tau) \cdot \theta^-$  con  $\tau = 0.01$  (un 1% por paso). Se llama *soft update* o promedio de Polyak. El resultado: un blanco que cambia despacio y no marea al entrenamiento.

## La tercera pieza: el buffer de experiencia

Las dos redes deciden y se corrigen, pero ¿de dónde salen los ejemplos con los que se corrigen? De una tercera pieza igual de importante: el **buffer de experiencia** (en inglés, *replay buffer*), una **libreta gigante donde el agente apunta cada jugada** que vive: el estado en que estaba, la acción que hizo, la recompensa que ganó y el estado al que pasó. En vez de aprender solo de la última jugada y olvidarla, guarda **100.000** y, en cada paso de entrenamiento, saca un **puñado al azar** para repasar.



#### Por qué importa repasar lo viejo

Si solo estudiaras lo último que te pasó, te obsesionarías con ello y olvidarías el resto.

Mezclar jugadas antiguas y recientes (muestreo aleatorio) **rompe esa obsesión** y hace que la red aprenda de forma estable. Es la diferencia entre empollar de memoria el último examen y repasar todo el temario.

No todos los algoritmos usan esta libreta. Los que reaprovechan datos viejos (DQN, SAC, World Model) sí; PPO, en cambio, usa lo recién jugado y lo tira. Veremos esa diferencia —*off-policy* frente a *on-policy*— en la anatomía.

#### En una frase

El RL es un bucle agente—entorno guiado por recompensas; el agente aprende una **política** apoyándose en estimaciones de **valor**, una red neuronal que **aproxima** esas funciones, y trucos de estabilidad como la **red objetivo**. Todo lo demás son variaciones sobre este esqueleto.

# Del juego a la red neuronal

El ciclo de vida completo de una iteración de entrenamiento, paso a paso.

Los cuatro algoritmos comparten la misma **maquinaria**. Este es el ciclo que se repite en cada paso de entrenamiento; el mismo esqueleto sirve para cualquier proyecto de aprendizaje por refuerzo: cambia el juego y el algoritmo, pero las piezas (entorno, agente, memoria, bucle, métricas) son siempre las mismas.

## Los cuatro inquilinos de esta maquinaria

En este laboratorio conviven **cuatro algoritmos** distintos. Todos usan el mismo esqueleto que verás en esta sección; se diferencian en **qué aprenden** y en **cómo guardan la experiencia**. Esta es la vista de pájaro antes de entrar al detalle de cada uno:

### DQN

BASADO EN VALOR · OFF-POLICY

Aprende **cuánto vale cada acción** en cada situación (un número por acción) y elige la de mayor valor. Guarda la experiencia en un **almacén grande** y la reutiliza muchas veces. Eficiente y sobrio.

### PPO

ACTOR-CRÍTICO · ON-POLICY

Aprende **directamente la estrategia**: con qué probabilidad conviene cada acción. Usa los datos recién jugados y los descarta. Es el más **robusto y predecible** de afinar.

### SAC

MÁXIMA ENTROPÍA · OFF-POLICY

Como PPO aprende una estrategia, pero además se premia por **mantener variedad** en sus decisiones. Eso le hace **explorar muy bien** y no encasillarse pronto.

### World Model

BASADO EN MODELO · DYNA-Q

Primero aprende un **simulador del propio juego** y luego entrena también **imaginando** partidas dentro de él. Así exprime al máximo cada dato real. El más ambicioso.

Esta tabla es solo el "quién es quién". Cada algoritmo tiene después su propio capítulo con su red, su función de pérdida, sus parámetros y sus resultados; y al final, una **tabla comparativa** de los cuatro cara a cara.

## 1 · Cómo arranca y se ejecuta el juego

El entorno es una **simulación pura**: no dibuja nada, solo calcula física. Vive en `entornoArkanoid.js`. Al crearse o reiniciarse coloca los 28 ladrillos (4 filas × 7 columnas), lanza la pelota con una dirección aleatoria y sitúa la pala. La clave de diseño es que la física es de **paso fijo**: cada llamada a `paso(accion)` avanza **exactamente un instante** de simulación. El control de velocidad no acelera la física (eso cambiaría la dinámica y rompería el aprendizaje): solo decide cuántos pasos se ejecutan por fotograma.

```
// Avanza un paso de simulación aplicando la acción del agente.
paso(accion) {
  if (this.estaTerminado()) return { recompensa: 0, done: true };
  this.pasos++;
  this.recompensaPaso = RECOMPENSAS.PASO;           // 0 por paso (no penaliza sobrevivir)
  this._aplicarAccion(accion);                       // mueve la pala ← · →
  this._moverPelota();                               // x += vx ; y += vy (paso fijo)
  this._colisionParedes();
  this._colisionPala();                             // +0.10 si devuelve la pelota
  this._colisionLadrillos();                         // +1.0 por ladrillo roto
  this._verificarFin();                              // -1.0 si pierde la pelota; +5.0 si limpia
  // el nivel
  this._aplicarShaping();                            // recompensa densa: acercarse a la pelota
  this.recompensaEpisodio += this.recompensaPaso;
  return { recompensa: this.recompensaPaso, done: this.estaTerminado() };
}
```

Una sola función concentra toda la regla del juego. Devuelve la **recompensa** de ese paso y si el episodio ha **terminado** (done).

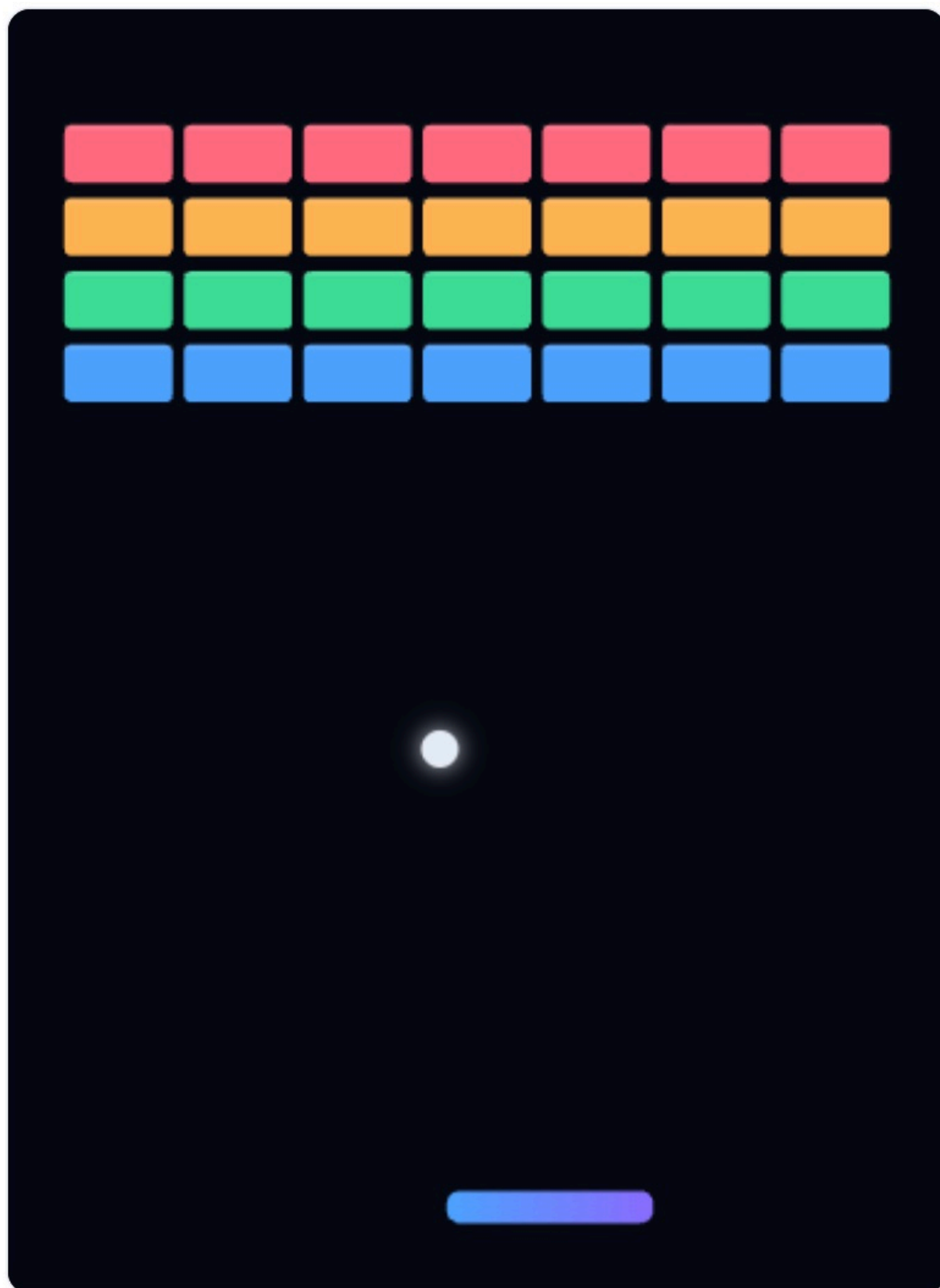


PARTIDA OBSERVADA



ENV 0

● jugando



Recompensa episodio: 0.02



Ladrillos: 0/28

Pasos: 1

La partida observada: pelota, pala (degradado azul-violeta) y los 28 ladrillos. Abajo, la recompensa del episodio, los ladrillos rotos y los pasos.

## El episodio: una partida con principio y fin

La unidad natural del aprendizaje por refuerzo es el **episodio: una partida completa**, desde que se saca la pelota hasta que esa partida acaba. Un episodio termina siempre de una de **tres maneras**:

### **Victoria**

El agente rompe los **28 ladrillos**. Recibe el premio grande (+5) y la partida acaba con éxito.

### **Pelota perdida**

La pelota se escapa por abajo porque la pala no llegó. Castigo (-1) y fin de la partida.

### **Tiempo agotado**

Pasan **600 pasos** sin ganar ni perder. Se corta la partida para no quedarse atascada eternamente.

En cuanto un episodio termina, ese entorno **se reinicia solo** y empieza otro inmediatamente: el agente nunca para de jugar. La **recompensa del episodio** es la suma de las recompensas de todos sus pasos, y es justo lo que el agente intenta maximizar. Por eso la métrica estrella es la **recompensa media de los últimos 100 episodios**: una sola partida es ruidosa (hay suerte), pero el promedio de 100 cuenta la verdad de si está mejorando.

### **PARA CURIOSOS: EL LÍMITE DE PASOS**

El tope está en `MAX_PASOS_EPISODIO = 600`. Sin él, un agente que aprendiera solo a "no perder" (devolver la pelota sin romper ladrillos) podría alargar una partida indefinidamente y bloquear el entrenamiento. El corte por tiempo es un final **neutro**: ni premio ni castigo extra, solo "se acabó el tiempo".

## Dos maneras de que el agente "vea" el juego

Para que una IA juegue hay **dos caminos**, y elegir bien marca la diferencia entre entrenar en horas o en semanas:

### **Leer la pantalla**

COMO UN HUMANO · DIFÍCIL

Darle los **píxeles** de la imagen y que aprenda a interpretarlos. Es lo más general (sirve para cualquier juego) pero carísimo: la red debe **primero** entender qué es una pelota mirando miles de imágenes, y **luego** aprender a jugar. Necesita redes enormes y millones de partidas.

### **Leer el estado interno**

NUESTRO ATAJO · EFICIENTE

Aprovechar que **nosotros programamos el juego** y conocemos sus variables. En vez de píxeles, le pasamos un puñado de **números que ya resumen la situación**. La red se ahorra "ver" y se concentra en decidir. Aprende muchísimo más rápido con redes pequeñas.

Hemos elegido el **segundo camino**: como el juego es nuestro, reducimos toda la pantalla a **6 variables**. Es el mismo principio que usar el marcador y la posición en vez de una foto del campo.

### **El estado: lo único que ve el agente**

El agente no ve píxeles, ve un vector de **6 números normalizados** a  $\sim[-1, 1]$ : posición x e y de la pelota, su velocidad ( $v_x, v_y$ ), la posición de la pala y la diferencia pelota-pala. Normalizar (que todo

viva en el mismo rango) es lo que permite que una red pequeña aprenda rápido.

src/entorno/entornoArkanoid.js · obtenerVectorEstado()

EL ESTADO

```
return [
  p.x * 2 - 1, p.y * 2 - 1, // posición de la pelota → [-1,1]
  p.vx / CFG.VELOCIDAD_PELOTA, p.vy / CFG.VELOCIDAD_PELOTA, // velocidad → [-1,1]
  this.pala.x * 2 - 1, // posición de la pala
  limitar((p.x - this.pala.x) * 2, -1, 1), // distancia horizontal pelota→pala
];
```

6 features. Pocas, pero suficientes para describir la situación. El reto del agente es mapear estos 6 números a la mejor de 3 acciones.

## 2 · Cómo enganchamos los controles: el agente "juega"

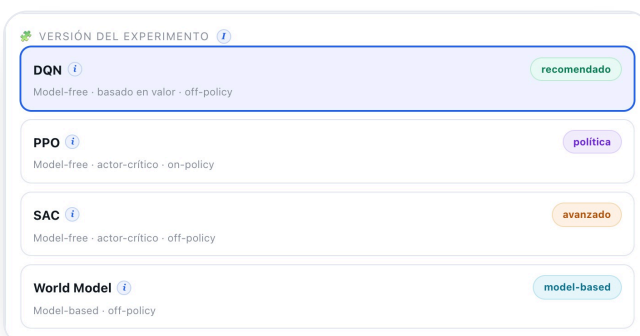
En un Arkanoid normal, un humano pulsa teclas. Aquí el **agente sustituye al jugador**: en cada paso, su red recibe el estado y devuelve la acción. Y como queremos aprender deprisa, no jugamos una partida, sino **cientos a la vez**: el estado de los N entornos se apila en un único tensor  $[N \times 6]$  y la red predice las N acciones **de una sola pasada** por la GPU (inferencia en lote). Esto es lo que hace que WebGPU rente: una llamada grande en vez de N pequeñas.

src/agentes/agenteDQN.js · seleccionarAcciones()

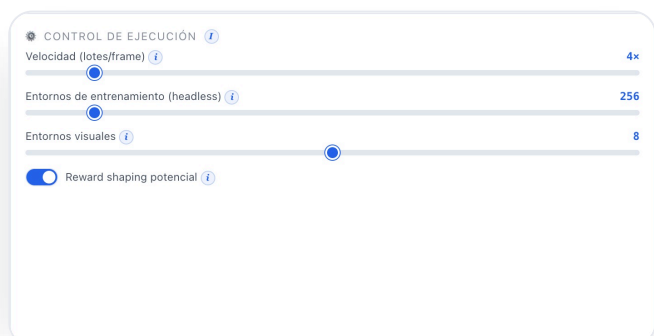
EL MANDO DEL AGENTE

```
seleccionarAcciones(estadosFlat, n, { entrenar = true } = {}) {
  const eps = entrenar ? this.epsilon : 0; // ε = probabilidad de explorar al azar
  const greedy = tf.tidy(() => {
    const sT = this._tensorEstados(estadosFlat, n); // tensor [N × 6]
    return this.redPolitica.predict(sT).argMax(1).dataSync(); // mejor acción de cada entorno
  });
  const acciones = new Int32Array(n);
  for (let i = 0; i < n; i++) // ε-greedy: a veces exploro, casi siempre exploto
    acciones[i] = Math.random() < eps ? (Math.random()*this.numAcciones)|0 : greedy[i];
  return acciones; // N acciones, una por partida
}
```

El agente convierte estados en acciones. **ε-greedy**: con probabilidad ε actúa al azar (explorar), si no elige la mejor acción conocida (explotar).



Eliges qué cerebro pilota: DQN, PPO, SAC o World Model. Cambiarlo recrea el agente desde cero.



Los mandos: velocidad (lotes/frame), nº de entornos de entrenamiento y de observación, y el reward shaping.

### 3 · Dos mundos: entrenar rápido y observar bonito

Si atáramos el aprendizaje a lo que se dibuja, estaríamos limitados a 8 ó 15 partidas. Por eso hay **dos pools de entornos** separados (`gestorEntornos.js`):

#### Pool headless

50–2000 envs · sin dibujar · generan los datos

+

#### Pool visual

8–15 envs · se dibujan · misma política

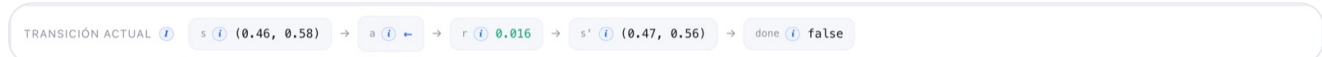
El pool **headless** (sin canvas) corre cientos de partidas para alimentar a la red. El pool **visual** ejecuta la **misma política** solo para que tú veas cómo juega. Es una ventana de observación, no el motor.



La rejilla de entornos visuales: muchas partidas en paralelo con la misma política. Puedes pinchar una para verla grande arriba.

### 4 · Cómo se guardan los datos para el entrenamiento

La unidad de experiencia es la **transición**:  $(s, a, r, s', done)$  — "estaba en el estado  $s$ , hice la acción  $a$ , gané  $r$ , acabé en  $s'$ , y el episodio terminó (o no)". Cada paso de cada entorno produce una. El gestor las empaqueta en **arrays planos** (muy eficientes para crear tensores) y se las pasa al agente para que las guarde.



La tira de transición muestra en vivo el  $(s \rightarrow a \rightarrow r \rightarrow s' \rightarrow done)$  del entorno observado: el ladrillo de información con el que se aprende.

¿Dónde se guardan? Depende de la familia del algoritmo:

#### Off-policy → Replay buffer

DQN, SAC y World Model guardan **100.000** transiciones en una memoria circular y entrenan muestreando lotes aleatorios. Reutilizan cada experiencia muchas veces y rompen la correlación temporal.

#### On-policy → Rollout buffer

PPO acumula un **rollout** (256 pasos × N entornos), calcula las ventajas con GAE, entrena unas pocas épocas y **descarta** los datos: solo valen mientras la política no cambie.

```
// Inserta un lote de transiciones en formato plano (buffer circular).
agregarLote(lote) {
  const n = lote.recompensas.length;
  for (let i = 0; i < n; i++) {
    const baseDst = this.pos * this.dim, baseSrc = i * this.dim;
    for (let k = 0; k < this.dim; k++) {
      this.s[baseDst + k] = lote.estados[baseSrc + k]; // estado s
      this.s2[baseDst + k] = lote.siguietes[baseSrc + k]; // estado siguiente s'
    }
    this.a[this.pos] = lote.acciones[i]; // acción a
    this.r[this.pos] = lote.recompensas[i]; // recompensa r
    this.done[this.pos] = lote.terminados[i]; // done
    this.pos = (this.pos + 1) % this.capacidad; // circular: pisa lo más viejo
    if (this.tam < this.capacidad) this.tam++;
  }
}
```

La experiencia se guarda en columnas (s, a, r, s', done) para poder muestrear lotes y convertirlos en tensores sin copias caras.

## 5 · El bucle de entrenamiento (la pieza que lo une todo)

El orquestador.js no sabe de algoritmos ni de pantallas: solo repite un ciclo. Esta es **una iteración** (un "lote"): se observan los estados, el agente decide, el juego avanza, se guarda la experiencia y el agente entrena. Es el corazón que late decenas de veces por segundo.

```
async ejecutarLote() {
  const n = gestor.numHeadless;
  const estados = gestor.obtenerEstadosEntrenamiento(); // 1· observar los N estados
  const acciones = this.agente.seleccionarAcciones(estados, n); // 2· la red decide N acciones
  const resultado = gestor.aplicarAcciones(acciones); // 3· el juego avanza N pasos
  this.agente.almacenarExperiencia(resultado); // 4· guardar (s,a,r,s',done)
  this.mtricas.registrarEpisodios(resultado.episodios); // 5· anotar episodios
  terminados
  const metrica = await this.agente.entrenar(); // 6· UN paso de aprendizaje
  this.pasoGlobal += n;
  // 7· cada 250 pasos: registrar una traza y avisar a la UID por el bus de eventos
}
```

Siempre los mismos 7 gestos. Cambiar de algoritmo solo cambia qué hace entrenar() por dentro; el bucle es idéntico.

Tres preguntas que aclaran cómo es realmente el entrenamiento:

### ¿Una partida o muchas?

**Muchas a la vez.** No jugamos una partida entera y luego aprendemos: en cada iteración avanzamos **un paso en cada uno de los cientos de entornos** en paralelo. Así, de un solo "latido" salen cientos de experiencias nuevas.

### ¿Cuándo aprende?

**Sin parar.** Después de cada paso colectivo, el agente hace **un** ajuste de la red. Jugar y aprender se intercalan continuamente, decenas de veces por segundo, no en fases separadas.

### ¿Cuándo paramos?

**Cuando tú quieras.** No hay un número mágico de partidas: el límite es el **tiempo** (o las iteraciones) que le dediques. Se entrena hasta que la curva de recompensa se aplanan y deja de mejorar.

## 6 · Cómo se entrena la red neuronal (el paso de gradiente)

Entrenar una red es ajustar sus **pesos** para que se equivoque menos. En RL no tenemos "respuestas correctas": construimos un **objetivo** a partir de la recompensa y de lo que la propia red cree que vale el futuro (ecuación de Bellman), y empujamos la predicción hacia ese objetivo. El truco para que no diverja es usar una **red objetivo**: una copia que se mueve despacio y nos da un blanco estable al que apuntar.



### PARA CURIOSOS: EL OBJETIVO

$\text{objetivo} = r + \gamma \cdot (1 - \text{done}) \cdot Q^-(s', a^*)$ . Es la recompensa de ahora más el valor (descontado por  $\gamma=0.99$ ) que la red objetivo  $Q^-$  asigna a la mejor acción del estado siguiente. Si el episodio termina (done), el término del futuro desaparece.

src/agentes/agenteDQN.js · entrenar()

EL APRENDIZAJE

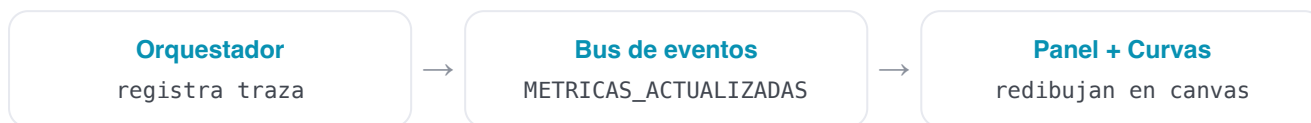
```
// El objetivo TD se calcula SIN gradiente (es el "blanco" estable).
const objetivo = tf.tidy(() => {
  const aStar = this.redPolitica.predict(s2T).argMax(1); // Double DQN: la red online elige a*
  const qSig = this.redObjetivo.predict(s2T)...sum(1); // la red objetivo la evalúa
  return rT.add(qSig.mul(gamma).mul(noTerm)); // r + γ · (1-done) · Q'(s', a*)
});
// Un paso de descenso de gradiente: acerca Q(s,a) al objetivo (pérdida Huber).
const lossT = this.optimizador.minimize(() => {
  const qa = this.redPolitica.predict(sT).mul(aOneHot).sum(1);
  return tf.losses.huberLoss(objetivo, qa);
}, true, this._varsPolitica);
// La red objetivo sigue a la online MUY despacio (Polyak): estabilidad.
actualizacionSuave(this.redPolitica, this.redObjetivo, tau); // τ = 0.01
```

Bellman + red objetivo + pérdida Huber + actualización suave. Cada algoritmo cambia esta receta, pero la idea —empujar la predicción hacia un objetivo— se mantiene.

**Higiene de memoria (TensorFlow.js).** Cada operación crea tensores en la GPU. Si no se liberan, la memoria crece sin parar. Todo el laboratorio usa `tf.tidy` y un gestor de tensores; el resultado es un recuento **plano** de tensores durante todo el entrenamiento (verificado: 26 en DQN, 76 en SAC...).

## 7 · Cómo se van actualizando las gráficas

El orquestador no toca la pantalla: emite **trazas** por un **bus de eventos**. La UI está suscrita y, cuando llega una traza nueva (cada 250 pasos), refresca las tarjetas de métricas y **redibuja las cuatro curvas** en sus <canvas>. Así el motor y la interfaz quedan desacoplados: podrías entrenar sin pantalla (lo hacemos en Node) o cambiar la UI sin tocar el algoritmo.



Así se ve la diferencia entre el arranque (sin datos) y tras unos segundos de entrenamiento real:



**Antes:** recién arrancado, las curvas están vacías (aún no hay episodios ni gradientes).

**Después:** la recompensa sube (verde), la pérdida se asienta (rojo), el buffer se llena (azul) y  $\epsilon$  decae (ámbar).

## Qué cuenta cada curva

El panel dibuja **cuatro curvas**. Las **dos primeras son iguales en los cuatro algoritmos**; las otras dos cambian porque miden el mecanismo propio de cada uno.

### ● Recompensa (todos)

La métrica reina: recompensa media de los últimos 100 episodios. **Si sube, aprende**. Es la única curva que de verdad dice si el agente mejora. La leemos en los cuatro modelos.

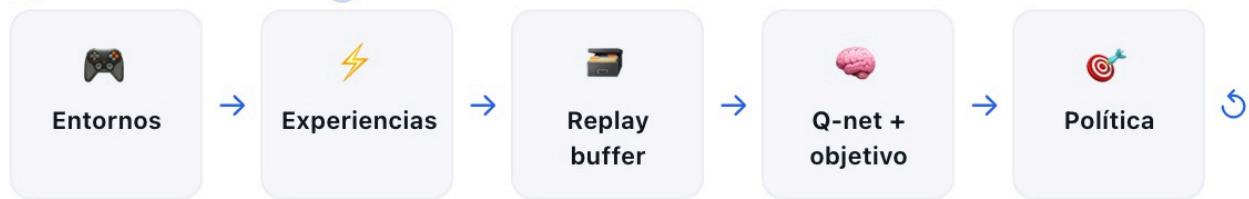
### ● Pérdida (todos)

El error de la red al ajustarse. **Ojo:** en RL el objetivo se mueve mientras aprendes, así que **no tiene que bajar siempre** como en otros campos. Lo que vigilamos es que **no se dispare** (eso sí sería mala señal).

Y estas son las **dos curvas propias** de cada algoritmo (las explicamos en detalle en su capítulo):

| Modelo      | Tercera curva                                                                                      | Cuarta curva                                                           |
|-------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| DQN         | <b>Buffer:</b> cómo se llena la memoria ( $0 \rightarrow 100.000$ ).                               | $\epsilon$ : el azar baja poco a poco; explora menos según aprende.    |
| PPO         | <b>Pérdida del crítico:</b> cómo de bien estima el valor de cada situación.                        | <b>Entropía:</b> cuánta variedad mantiene; baja despacio al decidirse. |
| SAC         | <b>Temperatura <math>\alpha</math>:</b> cuánto se premia a sí mismo por explorar (se autorregula). | <b>Entropía:</b> la variedad de sus decisiones.                        |
| World Model | <b>Error del modelo:</b> cuánto falla su simulador interno (baja = predice mejor).                 | $\epsilon$ : el azar de exploración, igual que en DQN.                 |

#### FLUJO DE DATOS



Off-policy: las experiencias se guardan en un replay buffer y se reutilizan en minibatches aleatorios, rompiendo la correlación temporal y mejorando la eficiencia de datos.

El propio laboratorio dibuja su flujo de datos, y se adapta al algoritmo (aquí, off-policy con replay buffer).

#### En una frase

Observar estados → la red decide acciones → el juego avanza y produce recompensas → se guardan transiciones → un paso de gradiente acerca la predicción a un objetivo de Bellman → se emite una traza que refresca las gráficas. Repite miles de veces por minuto. **Eso es entrenar un agente.**

# DQN — Deep Q-Network

Aprende cuánto vale cada acción y elige la de mayor valor.

## ¿Qué es DQN?

DQN aprende una función  $Q(s,a)$ : dado un estado, estima la **recompensa futura total** que cabe esperar si tomamos cada acción y luego seguimos jugando bien. Con esa tabla de valores, actuar es trivial: elige la acción de mayor Q. Lo "deep" es que esa función Q no es una tabla, sino una **red neuronal**, capaz de generalizar a estados que nunca vio exactamente.



### Una analogía

Como aprender a jugar a los dardos: cada tirada te da una puntuación (recompensa) y, con la práctica, intuyes cuánto "vale" apuntar a cada zona. DQN hace eso con números: estima el valor de cada movimiento y se queda con el mejor.

## ¿Para qué sirve? (en el mundo real)



### Videojuegos de Atari

DQN fue el primer algoritmo que aprendió a jugar decenas de juegos de Atari a nivel humano solo a partir de píxeles.



### Control discreto

Decisiones del tipo "encender/apagar/mantener": climatización, gestión de colas, semáforos.



### Recomendación

Elegir el siguiente contenido a mostrar para maximizar el interés a largo plazo.

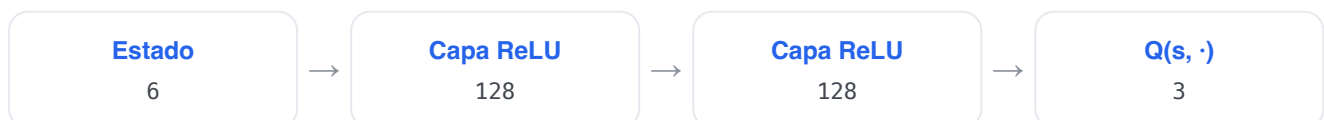


### Energía

Decidir cuándo cargar/descargar una batería según precio y demanda.

## Cómo funciona (su estructura)

DQN no guarda  $Q(s,a)$  en una tabla (imposible con un estado continuo): usa una **red neuronal para aproximar Q**. Es exactamente el MLP descrito en [Fundamentos · La red neuronal](#): entran los 6 números del estado, pasan por dos capas ocultas de 128 neuronas (ReLU) y salen **3 valores Q**, uno por acción. Para estabilizar el entrenamiento mantenemos **dos copias**: la **red actual** (decide y se entrena) y la **red objetivo** (copia lenta que fija el blanco de Bellman), tal y como se explica en Fundamentos.



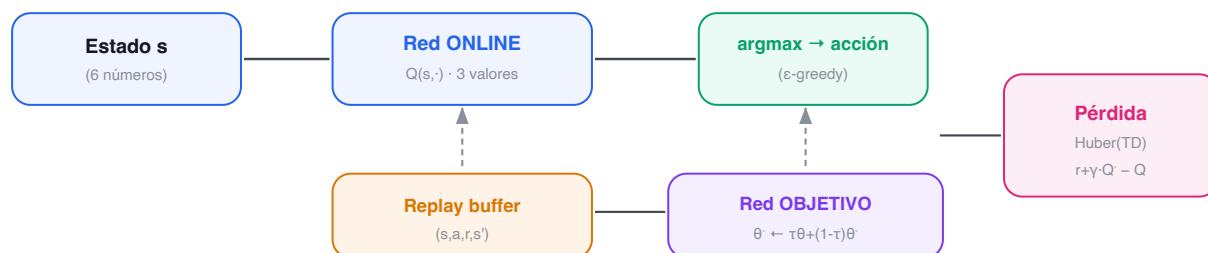
### PARA CURIOSOS: LA ECUACIÓN DE BELLMAN

$Q(s,a) \leftarrow r + \gamma \cdot \max_{a'} Q(s', a')$ . El valor de una acción se acerca a la recompensa inmediata más el mejor valor posible del estado siguiente.

## La idea clave: Bellman + estabilizadores

El valor de ahora se define en términos del valor del futuro. Para que ese "perseguirse a sí mismo" no diverja añadimos tres mejoras clásicas: **Double DQN** (separa quién elige y quién evalúa la acción), **pérdida Huber** (menos sensible a errores grandes) y **red objetivo** con actualización suave.

DQN: la red online decide y se entrena; la red objetivo da un blanco estable



El circuito de DQN: la red online elige la acción (ε-greedy); el replay buffer y la red objetivo alimentan la pérdida TD que la entrena.

## Ejemplo paso a paso: una decisión de DQN

1. En el estado actual, la red da  $Q = [0.2, 0.8, 0.5]$  para  $[\leftarrow, \cdot, \rightarrow]$ .
2. Con  $\epsilon = 0.1$ : el 90% de las veces elige el máximo ( $\cdot$ ,  $Q=0.8$ ); el 10%, una acción al azar (explorar).
3. Para entrenar (Double DQN): la red **online** dice que en el estado siguiente la mejor acción es « $\rightarrow$ »; la red **objetivo** la valora en 1.2. El objetivo es  $r + \gamma \cdot 1.2$ .
4. Separar quién **elige** (online) de quién **evalúa** (objetivo) evita que un Q inflado por ruido se realimente. Eso es Double DQN.

## Tres estabilizadores para que DQN no se rompa

Un DQN "ingenuo" tiende a desbocarse, porque persigue un objetivo que él mismo va moviendo. Tres mejoras clásicas lo domestican, y las tres están activas en este laboratorio:

### ① Double DQN

Un solo Q se autoengaña: como siempre se queda con el **máximo**, hereda el ruido del valor más optimista y acaba **sobreestimando**. La solución: la red **actual elige** cuál es la mejor acción del futuro y la red **objetivo le pone nota**. Dos opiniones distintas frenan el optimismo ciego.

### ② Pérdida Huber

En vez de penalizar el error **al cuadrado** (que agranda los fallos grandes), Huber es **suave con los errores grandes**. Así, un objetivo disparatado por ruido ya no zarandea a la red. El detalle, justo debajo.

### ③ Red objetivo (soft update)

El blanco lo fija una **copia lenta** de la red que solo se actualiza un **1% por paso** ( $\tau = 0.01$ ). Sin ella, perseguiríamos nuestra propia sombra y el entrenamiento divergería.

## La función de pérdida: ¿qué error minimizamos?

Entrenar la red consiste en **acortar la distancia** entre lo que predice ( $Q(s, a)$ ) y el objetivo de Bellman. Pero "distancia" se puede medir de varias formas, y elegir bien evita muchos disgustos:

| Opción                      | Cómo mide el error                   | Por qué la elegimos o no                                                                                        |
|-----------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>MSE (cuadrático)</b>     | Eleva el error al cuadrado.          | Un objetivo ruidoso —y los hay— genera un error enorme que <b>domina el ajuste</b> y desestabiliza. Descartada. |
| <b>MAE (absoluto)</b>       | El valor absoluto del error.         | Robusto, pero su empuje es constante: cerca del acierto <b>no afina</b> . Descartada.                           |
| <b>Huber ✓ (la nuestra)</b> | Cuadrático cerca de 0, lineal lejos. | Lo mejor de ambas: <b>afina de cerca</b> como MSE y <b>no se desboca</b> con errores grandes como MAE.          |

### PARA CURIOSOS: HUBER EN UNA LÍNEA

Para un error  $\delta = \text{objetivo} - Q(s, a)$ : si  $|\delta| \leq 1$  usa  $\frac{1}{2} \cdot \delta^2$  (cuadrático, afina); si  $|\delta| > 1$  usa  $|\delta| - \frac{1}{2}$  (lineal, no explota). En el código es `tf.losses.huberLoss`.

## El experimento: objetivo, entorno y parámetros

**El entorno.** Arkanoid: el agente observa 6 números, dispone de 3 acciones y recibe +1 por ladrillo, +0.1 por devolver la pelota y -1 por perderla. **El objetivo:** maximizar la recompensa acumulada por episodio.

| AJUSTE                                 | VALOR                  | QUÉ ES Y QUÉ EFECTO TIENE                                                                                        |
|----------------------------------------|------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>learning_rate</b>                   | 8e-4                   | El tamaño de cada paso de aprendizaje. Grande = rápido pero inestable; pequeño = lento pero estable.             |
| <b>gamma (<math>\gamma</math>)</b>     | 0.99                   | Cuánto importan las recompensas futuras frente a las inmediatas (0 = cortoplacista, $\rightarrow 1$ = previsor). |
| <b>epsilon (<math>\epsilon</math>)</b> | 1.0 $\rightarrow$ 0.05 | Probabilidad de explorar al azar; decae a lo largo de 25.000 pasos hacia 0.05 (casi siempre explotar).           |
| <b>replay buffer</b>                   | 100.000                | Cuántas experiencias recordamos para muestrear lotes aleatorios.                                                 |
| <b>batch_size</b>                      | 128                    | Cuántas transiciones se usan en cada paso de gradiente.                                                          |
| <b>tau (<math>\tau</math>)</b>         | 0.01                   | Velocidad a la que la red objetivo sigue a la principal (1% por paso): estabilidad.                              |

## ¿Por qué estos parámetros? (búsqueda en rejilla)

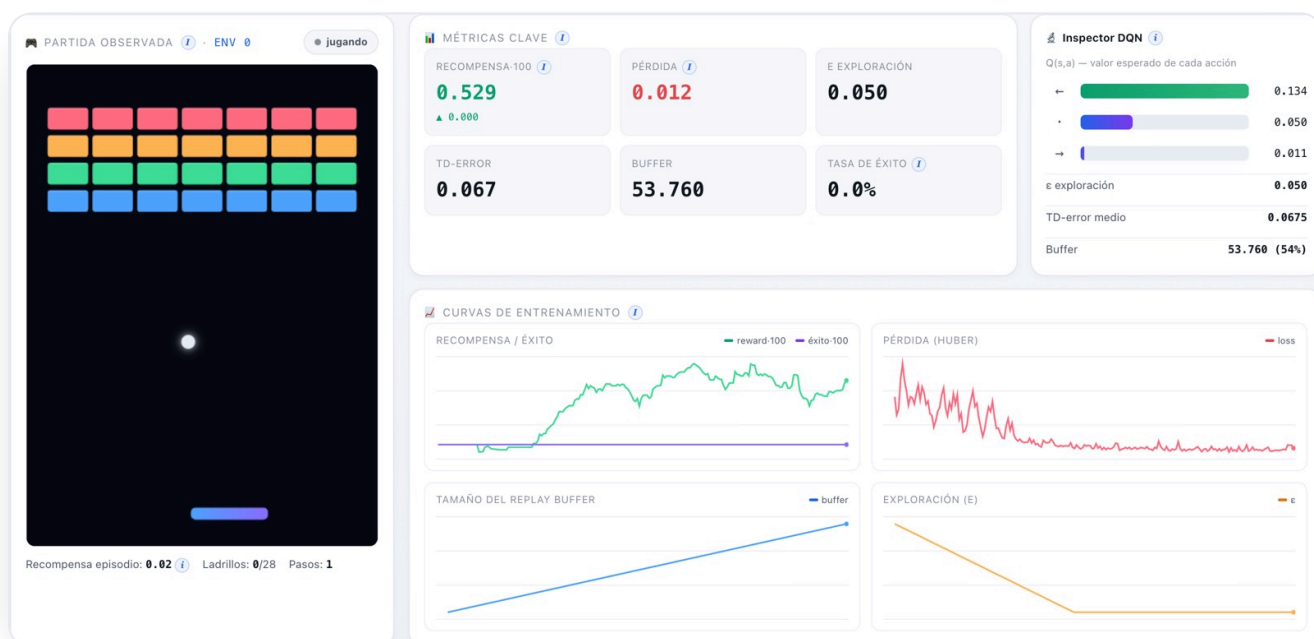
Los dos ajustes más delicados de DQN son el **ritmo de aprendizaje** (learning rate) y **cuánto pesa el futuro** ( $\gamma$ ). Para no elegirlos a ojo, barrimos una rejilla de 3×3 combinaciones; cada una entrenó 45.000 pasos y medimos la recompensa final alcanzada:

| LEARNING RATE     | $\Gamma = 0.97$ | $\Gamma = 0.99$ | $\Gamma = 0.995$ |
|-------------------|-----------------|-----------------|------------------|
| 3e-4 · lento      | 1.44            | 1.53            | 1.80             |
| 8e-4 · el nuestro | 1.88            | 2.41            | 1.62             |
| 1.5e-3 · rápido   | 2.62            | 1.26            | 2.23             |

El ritmo rápido (1.5e-3) logra el **pico más alto** (2.62)... pero también el **peor resultado** de toda la rejilla (1.26): es el más inestable, con una varianza enorme entre corridas. El lento (3e-4) aprende de forma consistente pero **se queda corto** (máximo 1.80). Nuestro valor (8e-4 con  $\gamma = 0.99$ ) alcanza **2.41**: el mejor resultado de la zona estable, sin las caídas bruscas del rápido. De ahí que sea el valor por defecto: **rendimiento alto y baja varianza**.

Cada celda es **una sola** corrida de 45.000 pasos, así que conviene leer estos números como **tendencias**, no como medias exactas: el RL es ruidoso (la fila del 1.5e-3 lo grita). Aun así, el patrón —el ritmo intermedio es el más fiable— es claro.

## La interfaz

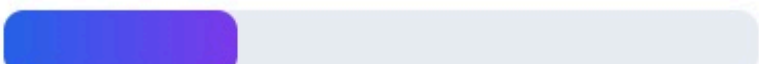


DQN entrenando: a la derecha, el inspector muestra los 3 valores Q (la acción greedy, en verde). Las curvas confirman que aprende: recompensa al alza,  $\epsilon$  decayendo.

## Inspector DQN

$Q(s,a)$  — valor esperado de cada acción

←  0.134

·  0.050

→  0.011

---

$\epsilon$  exploración 0.050

---

TD-error medio 0.0675

---

Buffer 53.760 (54%)

Inspector de DQN:  $Q(s,a)$  de cada acción, el  $\epsilon$  actual, el TD-error medio y el llenado del buffer.

## Resultados

En una ejecución real en el navegador (WebGPU), la recompensa media de los últimos 100 episodios sube de **~0.0 a ~0.9** en cuestión de segundos, con un ritmo de **~24.000 experiencias/s**. Con entrenamientos más largos (los 45.000 pasos de la rejilla anterior) la media **supera el 2**. El agente aprende con claridad a **seguir y devolver la pelota**; limpiar los 28 ladrillos (tasa de éxito  $> 0$ ) exige entrenamientos largos.



Curvas reales de DQN: recompensa·100 (verde) al alza, pérdida Huber (rojo) que se asienta, buffer (azul) llenándose y  $\epsilon$  (ámbar) decayendo.

### ⚠ El problema

Un DQN "de libro" **sobreestima** los valores Q (siempre coge el máximo, que arrastra el ruido) y la recompensa oscila o se desploma tras un buen pico.

### ✓ La mejora encontrada

**Double DQN** (la red online elige la acción y la objetivo la evalúa) + **Huber** + **soft update**. Resultado: aprendizaje más estable y sin fugas de memoria (26 tensores constantes).

## Interpretación: qué pasó al entrenar

### 📊 Qué observamos

- La recompensa sube rápido de  $\sim 0$  a  $\sim 0.9$  y luego asciende más despacio.
- Al decaer  $\epsilon$  y explotar más, la curva se vuelve **más estable**.
- La tasa de éxito (limpiar los 28 ladrillos) apenas despegga en runs cortas.

### ✓ Qué funcionó

- Aprende sin dudar a **seguir y devolver** la pelota.
- Double DQN + Huber + soft update: **cero divergencias**.
- Memoria **plana** (26 tensores) de principio a fin.

### ↗ Qué mejoraríamos

- Romper los últimos ladrillos pide **exploración más lista** que el puro azar  $\rightarrow$  ahí brillará SAC.
- Probar **repaso priorizado** (revisar más las jugadas sorprendentes).
- Entrenar más pasos: 45k se queda corto para subir la tasa de éxito.

## Conclusiones

DQN es el **primero** de los cuatro y hará de **línea de base**: los siguientes algoritmos los compararemos contra él. Su gran virtud es la **eficiencia con los datos** (reutiliza cada experiencia muchas veces) a cambio de necesitar varios estabilizadores para no romperse.

### En una frase

DQN es **off-policy** y **basado en valor**: aprende  $Q(s,a)$  reutilizando un **replay buffer**, explora con  **$\epsilon$ -greedy** y se estabiliza con una **red objetivo**. Es el cimiento del RL profundo y el mejor punto de partida.

# PPO — Proximal Policy Optimization

Optimiza la política directamente, pero con pasos prudentes.

## Primero: ¿qué es una "política"?

Una **política** es, sencillamente, la **estrategia** del agente: su "instinto" de qué hacer en cada situación. En la práctica es una función que, dado un estado, **reparte probabilidades** entre las acciones: "en esta posición, 60% moverme a la derecha, 30% quedarme quieto, 10% a la izquierda". Jugar es consultar ese instinto y dejarse llevar (a veces probando lo menos probable, para explorar). Esa estrategia se denomina política,  $\pi(a | s)$ .

**Política** La estrategia del agente: qué probabilidad asigna a cada acción en cada situación. Aprender, para PPO, es **reajustar esas probabilidades** para ganar más recompensa.

## ¿Qué es PPO?

DQN aprendía **cuánto vale** cada acción y deducía la jugada a partir de esos valores. PPO va **directo al grano**: ajusta la **estrategia misma**, subiendo la probabilidad de las acciones que salieron bien y bajando la de las que salieron mal. Para saber qué fue "bien" o "mal" se apoya en una **segunda red**, la **crítica**, que estima cómo de buena era la situación (su **valor**); al comparar lo que de verdad ocurrió con lo que la crítica esperaba se obtiene la **ventaja**: cuánto mejor (o peor) salió la acción de lo previsto. PPO es hoy el **caballo de batalla** del RL: estable, robusto y fácil de afinar.



### Una analogía

Como mejorar tu técnica de escalada poco a poco: si cambias tu estilo de golpe, te caes. PPO da **pasos pequeños y controlados** sobre la política para no estropear lo ya aprendido.

## ¿Para qué sirve? (en el mundo real)



### Robótica y control

Acciones continuas o discretas en robots, brazos y locomoción; es el algoritmo por defecto en muchas suites.



### RLHF en LLMs

PPO se usó para alinear modelos de lenguaje con preferencias humanas (la "P" de ChatGPT entrenado con RLHF).



### Juegos complejos

Dota 2 (OpenAI Five) y muchos entornos de simulación se entrenaron con PPO a gran escala.



### Optimización de procesos

Control de sistemas donde la estabilidad del entrenamiento es crítica.

## 📌 Cómo funciona (su estructura)

Dos redes MLP independientes: el **actor** ( $6 \rightarrow 128 \rightarrow 128 \rightarrow 3 \text{ logits} \rightarrow \text{softmax} = \text{probabilidades}$ ) y la **crítica** ( $6 \rightarrow 128 \rightarrow 128 \rightarrow 1 = \text{valor del estado}$ ).



**Actor-crítico: el actor decide, el crítico evalúa cuánto vale el estado**



Estructura actor-crítico: el actor propone acciones (probabilidades) y el crítico estima el valor del estado; con ambos se calcula la ventaja.



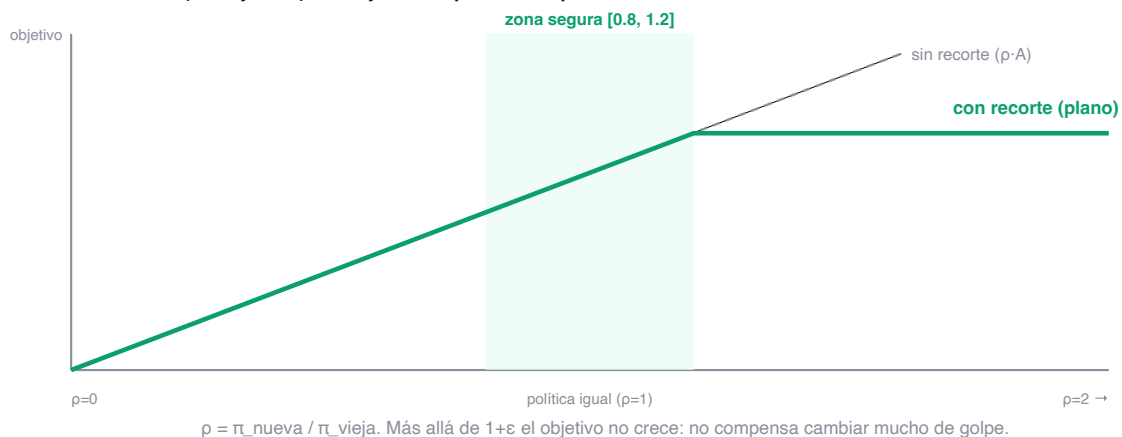
### PARA CURIOSOS: EL OBJETIVO RECORTADO

$L = E[ \min( p \cdot A, \text{clip}(p, 1-\epsilon, 1+\epsilon) \cdot A ) ]$ , donde  $p$  es el cociente entre la política nueva y la vieja y  $A$  es la ventaja. El recorte impide que  $p$  se aleje de 1 más de  $\epsilon$  (con  $\epsilon=0.2$ , fuera de  $[0.8, 1.2]$ ).

### La idea clave: el recorte (clip)

PPO mide cuánto cambia la política (ratio  $p$ ) y **recorta** ese cambio a  $[1-\epsilon, 1+\epsilon]$  (con  $\epsilon=0.2$ , a  $[0.8, 1.2]$ ). Así nunca da un paso tan grande que arruine la política. Las ventajas se estiman con **GAE** (combina errores TD de varios pasos) para reducir la varianza.

**El recorte de PPO (ventaja  $A>0$ ): la mejora se 'aplana' si la política cambia demasiado**



El recorte en acción: si la nueva política se aleja demasiado de la vieja ( $p$  fuera de  $[0.8, 1.2]$ ), el objetivo deja de crecer. Eso frena los cambios bruscos.



### Ejemplo paso a paso: el recorte de PPO

1. El agente tomó «→» y resultó mejor de lo esperado: ventaja  $A = +2$ .

2. La política nueva hace esa acción un 30% más probable que la vieja  $\rightarrow$  ratio  $p = 1.3$ .
3. Objetivo sin recortar:  $p \cdot A = 1.3 \cdot 2 = 2.6$ . Pero  $p=1.3$  supera el límite  $1+\epsilon = 1.2 \rightarrow$  se recorta a  $1.2 \cdot 2 = 2.4$ .
4. PPO toma el **mínimo**:  $\min(2.6, 2.4) = 2.4$ . Así, por mucho que mejore, no deja que la política se mueva más de un 20% de golpe: estabilidad.

## La función de pérdida: por qué el recorte

La idea de "subir la probabilidad de lo que salió bien" tiene una trampa: si te pasas, **destrozas** la política de un solo paso. Había tres formas de evitarlo, y elegimos la tercera:

| Opción                                | Qué hace                                                        | Por qué la elegimos o no                                                                         |
|---------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Gradiente "a pelo" (REINFORCE)</b> | Empuja cada acción según su ventaja, sin freno.                 | Un lote afortunado provoca un <b>paso gigante</b> que arruina lo aprendido. Demasiado inestable. |
| <b>TRPO</b>                           | Limita el cambio con una restricción matemática <b>exacta</b> . | Funciona, pero es <b>caro y enrevesado</b> de implementar (optimización con restricciones).      |
| <b>PPO-clip ✓ (la nuestra)</b>        | <b>Recorta</b> el ratio para que el paso nunca sea grande.      | Una aproximación <b>barata y simple</b> a TRPO: casi todo el beneficio, una fracción del coste.  |

Esa pérdida del actor no va sola. La pérdida total que minimiza PPO **suma tres términos**: el objetivo recortado (mejora la política con freno), el error de la **crítica** (que aprenda a estimar bien el valor) y un premio a la **entropía** (para no dejar de explorar).

### PARA CURIOSOS: LOS TRES TÉRMINOS

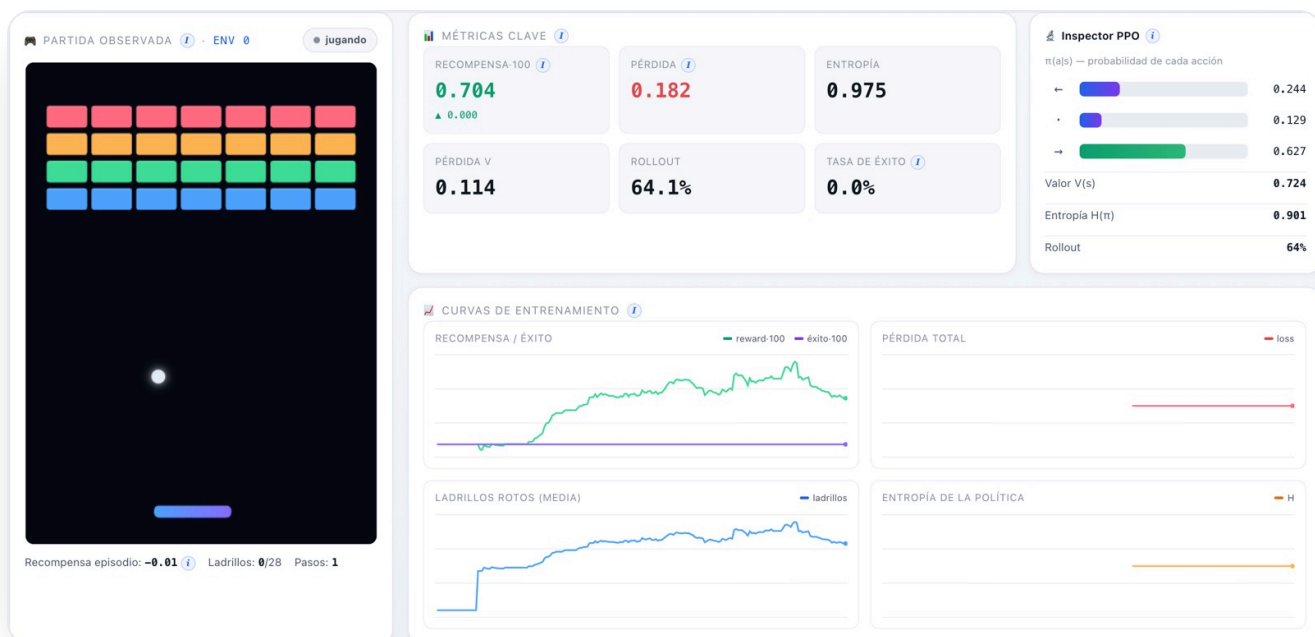
$L = L_{\text{clip}} - c_1 \cdot (V - \text{retorno})^2 + c_2 \cdot \text{entropía}$ . El primero es el objetivo recortado; el segundo, el error cuadrático de la crítica (con  $c_1 = \text{coef\_valor} = 0.5$ ); el tercero premia la variedad (con  $c_2 = \text{coef\_entropía} = 0.01$ ). Se restan/suman para optimizar todo a la vez.

## El experimento: objetivo, entorno y parámetros

Mismo entorno Arkanoid. PPO es **on-policy**: recoge un rollout, lo exprime en varias épocas y lo tira.

| AJUSTE                            | VALOR | QUÉ ES Y QUÉ EFECTO TIENE                                                                          |
|-----------------------------------|-------|----------------------------------------------------------------------------------------------------|
| <b>longitud_rollout</b>           | 256   | Pasos por entorno que se recogen antes de cada actualización.                                      |
| <b>épocas</b>                     | 4     | Cuántas veces se reaprovecha cada rollout antes de descartarlo.                                    |
| <b>clip <math>\epsilon</math></b> | 0.2   | Cuánto se permite cambiar la política por actualización (el "freno").                              |
| <b>GAE <math>\lambda</math></b>   | 0.95  | Equilibrio sesgo/varianza al estimar la ventaja (0 = solo 1 paso; $\rightarrow 1$ = muchos pasos). |
| <b>coef_entropía</b>              | 0.01  | Premia mantener algo de aleatoriedad en la política (exploración).                                 |
| <b>max_norma_grad</b>             | 0.5   | Recorta el gradiente para que ningún paso sea desmesurado.                                         |

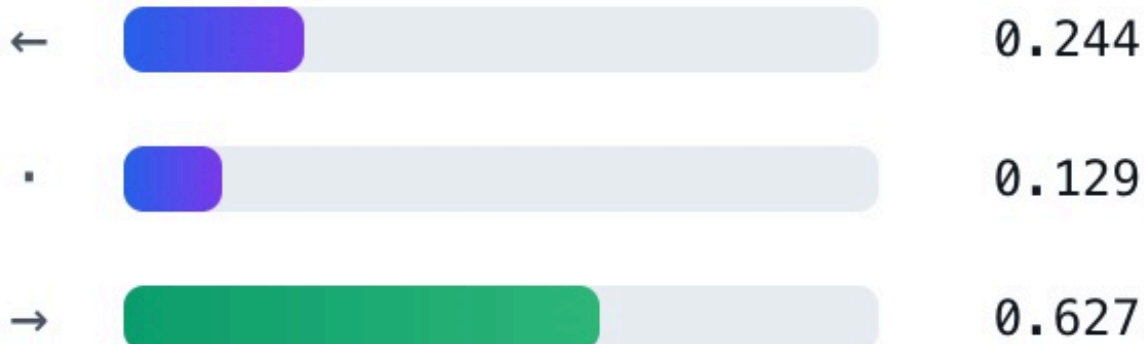
## La interfaz



PPO entrenando: el inspector muestra  $\pi(a|s)$  como barras de probabilidad, el valor  $V(s)$  y la entropía. Las curvas se adaptan (entropía en vez de  $\epsilon$ ).

## Inspector PPO

$\pi(a|s)$  — probabilidad de cada acción



Valor  $V(s)$  **0.724**

Entropía  $H(\pi)$  **0.901**

Rollout **64%**

Inspector de PPO: probabilidades de cada acción, valor estimado del estado y entropía de la política.

## Resultados

PPO aprende de forma muy **estable** (curva monótona). En ejecución real, la recompensa pasó de  $\sim 0.1$  a  $\sim 0.99$  en el navegador, y en una corrida más larga en CPU alcanzó  $\sim 4.08$  (ya rompe varios ladrillos por episodio). La entropía se mantiene alta al principio y baja despacio según la política se afila.



Curvas reales de PPO. La gráfica de "datos" muestra los ladrillos rotos de media; la de exploración, la entropía de la política.

### ⚠ El problema

Optimizar la política sin freno provoca pasos enormes que **destrozan** lo aprendido: la recompensa colapsa de golpe.

### ✓ La mejora encontrada

El **objetivo recortado + normalizar las ventajas + recorte de gradiente**. Resultado: la curva más estable de los cuatro, sin caídas bruscas.

## Interpretación: qué pasó al entrenar

### 📊 Qué observamos

- Curva **monótona**: sube sin las caídas bruscas de DQN.
- Reward ~0.99 en el navegador; ~4.08 en una corrida larga en CPU.
- La entropía arranca alta y **baja despacio** al afilarse la política.

### ✓ Qué funcionó

- El **recorte** evita el colapso: es el más estable de los cuatro.
- Llega a romper **varios ladrillos** por episodio.
- Apenas necesita afinado: funciona "a la primera".

### ↗ Qué mejoraríamos

- Al ser **on-policy** tira lo jugado: necesita **más experiencias** que DQN para lo mismo.
- Probar **rollouts más largos** y bajar el ritmo de aprendizaje al final.

## Comparación con el modelo anterior (DQN)

| Eje                 | DQN (01)                            | PPO (02)                                            |
|---------------------|-------------------------------------|-----------------------------------------------------|
| Qué aprende         | El <b>valor</b> de cada acción.     | La <b>estrategia</b> (probabilidades) directamente. |
| Memoria             | Replay buffer (reutiliza lo viejo). | Rollout: usa y <b>tira</b> .                        |
| Estabilidad         | Buena, pero con picos y caídas.     | <b>La mejor</b> : curva suave.                      |
| Eficiencia en datos | <b>Alta</b> (off-policy).           | Menor (on-policy).                                  |
| Afinado             | Sensible (varios estabilizadores).  | <b>Fácil</b> , robusto.                             |

En resumen: DQN exprime más cada dato, pero PPO es **más estable y más fácil de manejar**. Si DQN era el cimiento, PPO es la opción "sin sustos".

## I Conclusiones

### En una frase

PPO es **on-policy** y **actor-crítico**: mejora la política a pasitos **recortados**, usa la **ventaja** (GAE) para saber qué acciones reforzar y premia la **entropía** para no dejar de explorar. Estable y de uso general.

# SAC — Soft Actor-Critic (discreto)

Maximiza recompensa Y aleatoriedad, y se autorregula.

## ¿Qué es SAC?

SAC es actor-crítico **off-policy** con una idea extra: no solo maximiza la recompensa, también la **entropía** de la política (cuán aleatoria es). Esto mantiene la exploración viva y evita que el agente se "cierre" demasiado pronto en una sola estrategia. Usa **dos críticos** y un coeficiente de temperatura  **$\alpha$**  que **se ajusta solo**.



### Una analogía

Como un chef que ya tiene un plato que funciona pero sigue probando variaciones por si alguna es mejor: SAC se reserva, a propósito, una dosis de "probar cosas nuevas" mientras le compense.

## ¿Para qué sirve? (en el mundo real)



### Robótica continua

Es uno de los mejores algoritmos para control de robots reales por su eficiencia de muestra y robustez.



### Conducción y navegación

Entornos donde explorar de forma segura y constante es clave.



### Control fino

Tareas con muchas soluciones parecidas, donde mantener variedad ayuda a no estancarse.

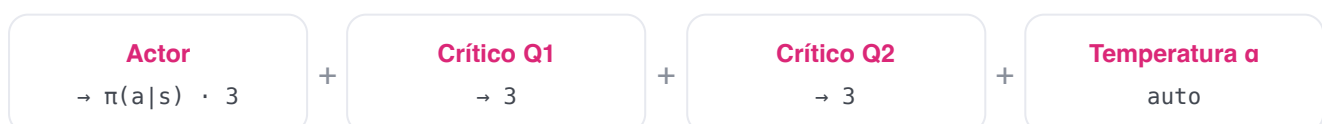


### Investigación

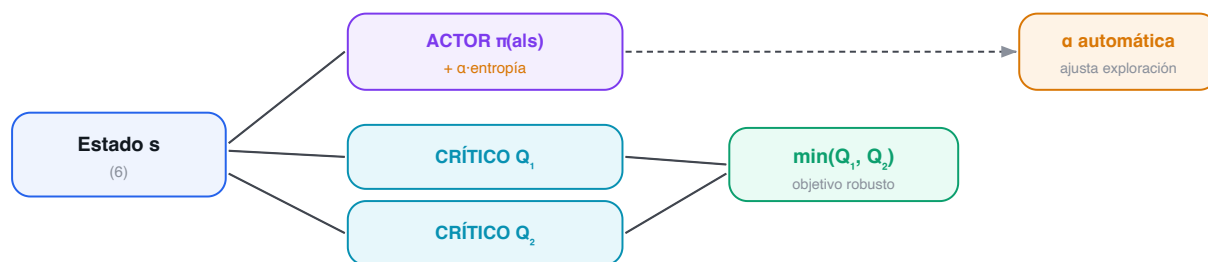
Banco de pruebas habitual para estudiar exploración y estabilidad.

## Cómo funciona (su estructura)

Un actor (política) y **dos críticos** Q (más sus dos redes objetivo). Trabajar con el **mínimo** de los dos críticos evita la sobreestimación de valores.



SAC: dos críticos (se usa el menor, evita sobreestimar) + entropía con temperatura  $\alpha$



SAC: un actor estocástico, dos críticos (se usa el menor para no sobreestimar) y una temperatura  $\alpha$  que ajusta sola cuánta exploración mantener.



#### PARA CURIOSOS: EL OBJETIVO DEL CRÍTICO

$y = r + \gamma \cdot \sum_a \pi(a|s') \cdot [\min(Q_1', Q_2') - \alpha \cdot \log \pi(a|s')] ]$ . Usa el menor de los dos críticos (para no sobreestimar) y resta  $\alpha \cdot \log \pi$ , el término que premia la variedad (entropía).

### La idea clave: temperatura automática

$\alpha$  pondera cuánta entropía exigimos. En vez de fijarlo a mano, SAC lo **aprende** para alcanzar una entropía objetivo (aquí,  $0.55 \cdot \log 3$ ). Si la política explora poco,  $\alpha$  sube; si explora demasiado, baja. En nuestras ejecuciones,  $\alpha$  bajó de 0.20 a **~0.165** sola.



#### Ejemplo paso a paso: cómo se autoajusta $\alpha$

1. La política da probabilidades  $[0.5, 0.3, 0.2]$ . Su entropía es  $H = -\sum p \cdot \log p \approx 1.03$  (cerca del máximo,  $\log 3 \approx 1.10$ ).
2. La entropía objetivo es  $0.55 \cdot \log 3 \approx 0.60$ .
3. Como  $1.03 > 0.60$ , el agente explora **más** de lo deseado  $\rightarrow \alpha$  **baja** (menos premio a la aleatoriedad) y la política se afila poco a poco.
4. Si fuera al revés (política casi fija,  $H < 0.60$ ),  $\alpha$  **subiría** para forzar más exploración. Por eso  $\alpha$  no hay que tocarla a mano: se ajusta sola.

## La función de pérdida: tres a la vez (y por qué dos críticos)

SAC no minimiza una pérdida, sino **tres a la vez**, una por cada cosa que aprende: los **críticos** (que estimen bien el valor), el **actor** (que mejore la estrategia buscando recompensa y variedad) y la **temperatura  $\alpha$**  (que se autoajuste). La decisión de diseño más importante es usar **dos** críticos en vez de uno:

| Opción                         | Qué hace                                          | Por qué la elegimos o no                                                                   |
|--------------------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------|
| Un crítico                     | Una sola red estima el valor.                     | Tiende a <b>sobreestimar</b> : se cree los valores optimistas y los realimenta. Inestable. |
| Dos críticos, usar el mínimo ✓ | Dos redes; se confía en la <b>más pesimista</b> . | Pesimista a propósito: no se traga los valores inflados. Es lo que da estabilidad a SAC.   |

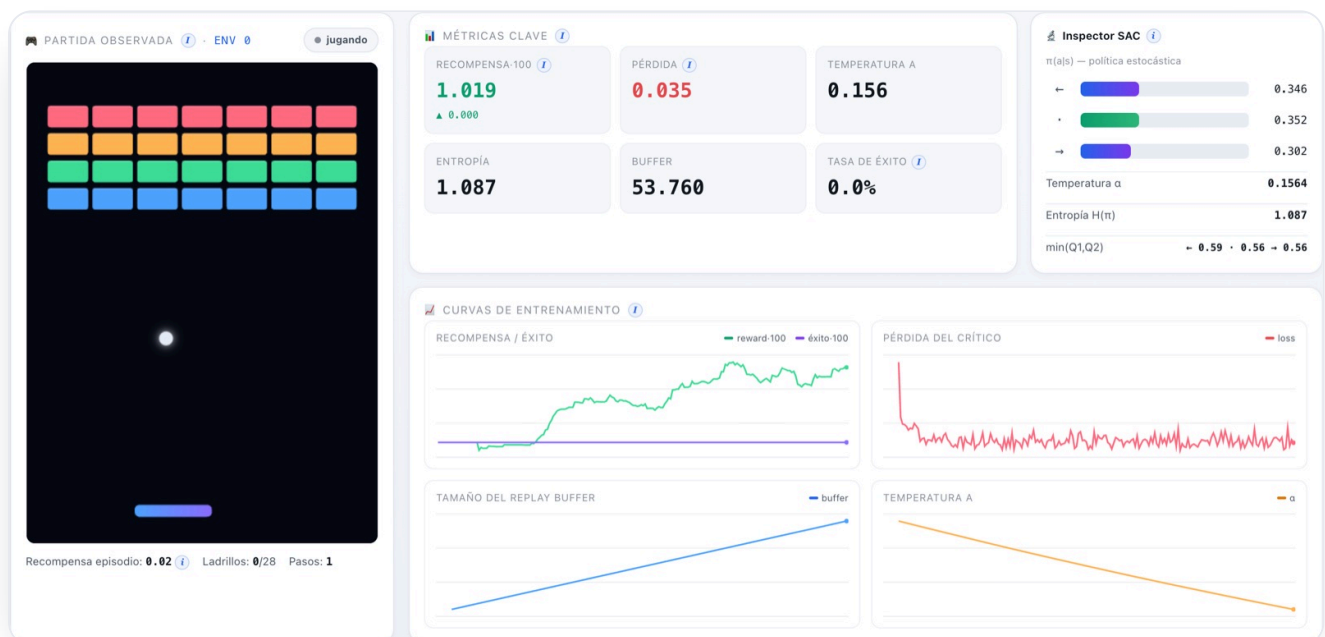
### 🐛 PARA CURIOSOS: LAS TRES PÉRDIDAS

**Crítico:** acercar cada Q al objetivo y de arriba (con el min de los dos). **Actor:** maximizar  $\sum \pi \cdot [\min(Q1, Q2) - \alpha \cdot \log \pi]$  (recompensa esperada + variedad).  **$\alpha$ :** ajustar la temperatura para acercar la entropía a su objetivo. Tres optimizadores, un solo agente.

## El experimento: objetivo, entorno y parámetros

| AJUSTE                                          | VALOR              | QUÉ ES Y QUÉ EFECTO TIENE                                                     |
|-------------------------------------------------|--------------------|-------------------------------------------------------------------------------|
| <b>lr_actor / crítico / <math>\alpha</math></b> | 6e-4 / 8e-4 / 6e-4 | Tres ritmos de aprendizaje, uno por red. El crítico aprende algo más rápido.  |
| <b>tau (<math>\tau</math>)</b>                  | 0.01               | Velocidad de las dos redes objetivo (soft update).                            |
| <b>entropía objetivo</b>                        | 0.55 · log 3       | Cuánta aleatoriedad queremos mantener; gobierna hacia dónde empuja $\alpha$ . |
| <b>batch / buffer</b>                           | 128 / 100.000      | Es off-policy: reutiliza experiencias de un replay buffer.                    |

## La interfaz



SAC entrenando: el inspector muestra la política, la temperatura  $\alpha$  y la entropía. La curva de "exploración" sigue a  $\alpha$  en vez de a  $\epsilon$ .

## Inspector SAC

$\pi(a|s)$  — política estocástica

←  0.346

·  0.352

→  0.302

---

Temperatura  $\alpha$  0.1564

---

Entropía  $H(\pi)$  1.087

---

$\min(Q1, Q2)$  ← 0.59 · 0.56 → 0.56

Inspector de SAC:  $\pi(a|s)$ , temperatura  $\alpha$  (auto-ajustada), entropía y el  $\min(Q1, Q2)$  por acción.

## Resultados

SAC aprende y la temperatura se estabiliza sola. En el navegador, la recompensa subió de  $\sim 0.0$  a  $\sim 0.99$ , con  $\alpha$  descendiendo de 0.20 a  $\sim 0.165$  y la entropía manteniéndose alta (exploración sana). Sin fugas: 76 tensores constantes durante todo el entrenamiento.



Curvas reales de SAC. La gráfica de exploración muestra la temperatura  $\alpha$  descendiendo poco a poco.

### ⚠ El problema

SAC discreto puede **diverger** (los Q se disparan) o quedarse demasiado aleatorio/demasiado rígido si  $\alpha$  está mal puesto a mano.

### ✓ La mejora encontrada

**Doble crítico** con mínimo (anti-sobreestimación) +  **$\alpha$  automática** (no hay que afinarla). Resultado: aprendizaje estable y exploración auto-regulada.

## Interpretación: qué pasó al entrenar

### 📊 Qué observamos

- La recompensa sube a  $\sim 0.99$  y la **temperatura  $\alpha$  baja sola** de 0.20 a  $\sim 0.165$ .
- La entropía se mantiene **alta**: explora sano, no se encasilla.
- Memoria plana: 76 tensores constantes (más que DQN: tiene más redes).

### ✓ Qué funcionó

- El **doble crítico** evitó que los valores se dispararan.
- La  **$\alpha$  automática** ahorró el ajuste más delicado a mano.
- Exploración constante sin perder estabilidad.

### ↗ Qué mejoraríamos

- Es el **más pesado**: actor + 2 críticos + 2 objetivos. Más cómputo por paso.
- Ajustar la **entropía objetivo** ( $0.55 \cdot \log 3$ ) según la fase del entrenamiento.
- Runs más largas para traducir su buena exploración en romper más ladrillos.

## Comparación con el modelo anterior (PPO)

| Eje                 | PPO (02)                   | SAC (03)                                                                 |
|---------------------|----------------------------|--------------------------------------------------------------------------|
| Familia             | Actor-crítico on-policy.   | Actor-crítico <b>off-policy</b> .                                        |
| Cómo explora        | Premio fijo a la entropía. | Premio a la entropía con <b><math>\alpha</math> que se autorregula</b> . |
| Memoria             | Rollout (usa y tira).      | Replay buffer ( <b>reutiliza</b> ).                                      |
| Coste por paso      | Bajo (2 redes).            | <b>Alto</b> (5 redes).                                                   |
| Eficiencia en datos | Menor.                     | <b>Alta</b> .                                                            |

SAC es como un PPO que **no tira la experiencia** y que **gestiona la exploración por sí mismo**: aprende con menos partidas a cambio de más cálculo por paso. Donde PPO es sencillo y estable, SAC es eficiente y curioso.

## I Conclusiones

### En una frase

SAC es **off-policy** y de **máxima entropía**: usa **dos críticos** para no sobreestimar y una **temperatura  $\alpha$  automática** para mantener la exploración justa. Muy eficiente en datos y robusto.

# World Model — Dyna-Q

Aprende un modelo del juego y entrena "imaginando".

## ¿Qué es un World Model?

Los tres anteriores son **model-free**: aprenden qué hacer sin entender el entorno. Un World Model es **model-based**: aprende un **modelo de la dinámica** —una red que predice, dado  $(s, a)$ , cuál será el siguiente estado  $s'$ , la recompensa  $r$  y si termina— y lo usa para generar experiencia **imaginada** con la que entrenar (Dyna-Q). Así exprime cada paso real muchas veces.



### Una analogía

Como un ajedrecista que, además de jugar, **imagina jugadas en su cabeza** para practicar sin mover ficha. El World Model "sueña" partidas con su modelo del mundo y aprende también de esos sueños.

## ¿Para qué sirve? (en el mundo real)



### Cuando los datos son caros

Robots reales o procesos donde cada paso cuesta tiempo/dinero: imaginar multiplica los datos disponibles.



### Planificación

Modelos del mundo (MuZero, Dreamer) que planifican mirando hacia delante con su propio simulador aprendido.



### Eficiencia de muestra

Aprender buenas políticas con muchos menos episodios reales.



### Simulación aprendida

Sustituir un simulador caro por una red rápida que lo aproxima.

## Cómo funciona (su estructura)

Dos piezas: un **Q-net** (como DQN) que decide, y un **modelo de dinámica** ( $6+3$  entradas  $\rightarrow 200 \rightarrow 200 \rightarrow \Delta s, r, \text{done}$ ) que predice el futuro. El modelo predice el **incremento**  $\Delta s$  (más estable que  $s'$  directo):  $s' = s + \Delta s$ .



### World Model (Dyna-Q): el agente aprende de experiencia REAL y también de experiencia IMAGINADA



Dyna-Q: la experiencia real entrena el modelo y el Q-net; además el modelo “imagina” experiencias extra que también entrenan el Q-net (5 por cada paso real).

Por cada paso real: se entrena el modelo de dinámica y el Q-net con datos reales; y, además, se hacen **5 actualizaciones** del Q-net con transiciones imaginadas por el modelo (planning).

#### La idea clave: planning con datos imaginados

Por cada paso real hacemos **5** actualizaciones del Q-net con transiciones que **genera el modelo**, partiendo de estados muestreados del buffer. Si el modelo es bueno, es casi gratis multiplicar el aprendizaje.

#### 🎲 Ejemplo paso a paso: una transición imaginada

1. Se toma un estado real del buffer; el agente (greedy) elegiría «→».
2. El **modelo** predice: la bola sube un poco ( $\Delta s$ ), recompensa  $r \approx 0$  (aún no rompe nada),  $\text{done} = \text{no}$ .
3. Con  $s' = s + \Delta s$  ya tenemos una transición **imaginada** ( $s, \rightarrow, 0, s', \text{no}$ ), sin haber tocado el juego real.
4. El Q-net entrena con ella igual que con una real (Bellman). Y esto se repite **5 veces por cada paso real** → 5x más aprendizaje... siempre que el modelo acierte (por eso vigilamos su error RMSE).

## La función de pérdida: dos aprendizajes, dos pérdidas

El World Model aprende **dos cosas distintas** a la vez, así que tiene **dos pérdidas**:

### ① El modelo de dinámica (supervisado)

Aprende a **predecir la física**: dado  $(s, a)$ , acertar el siguiente estado, la recompensa y si termina. Es un problema supervisado clásico: minimizar el **error entre lo predicho y lo real** (regresión).

### ② El Q-net (igual que DQN)

Decide las acciones y se entrena **exactamente como DQN**: objetivo de Bellman + Huber + red objetivo. La novedad no es cómo aprende, sino **con qué datos**: reales e imaginados.

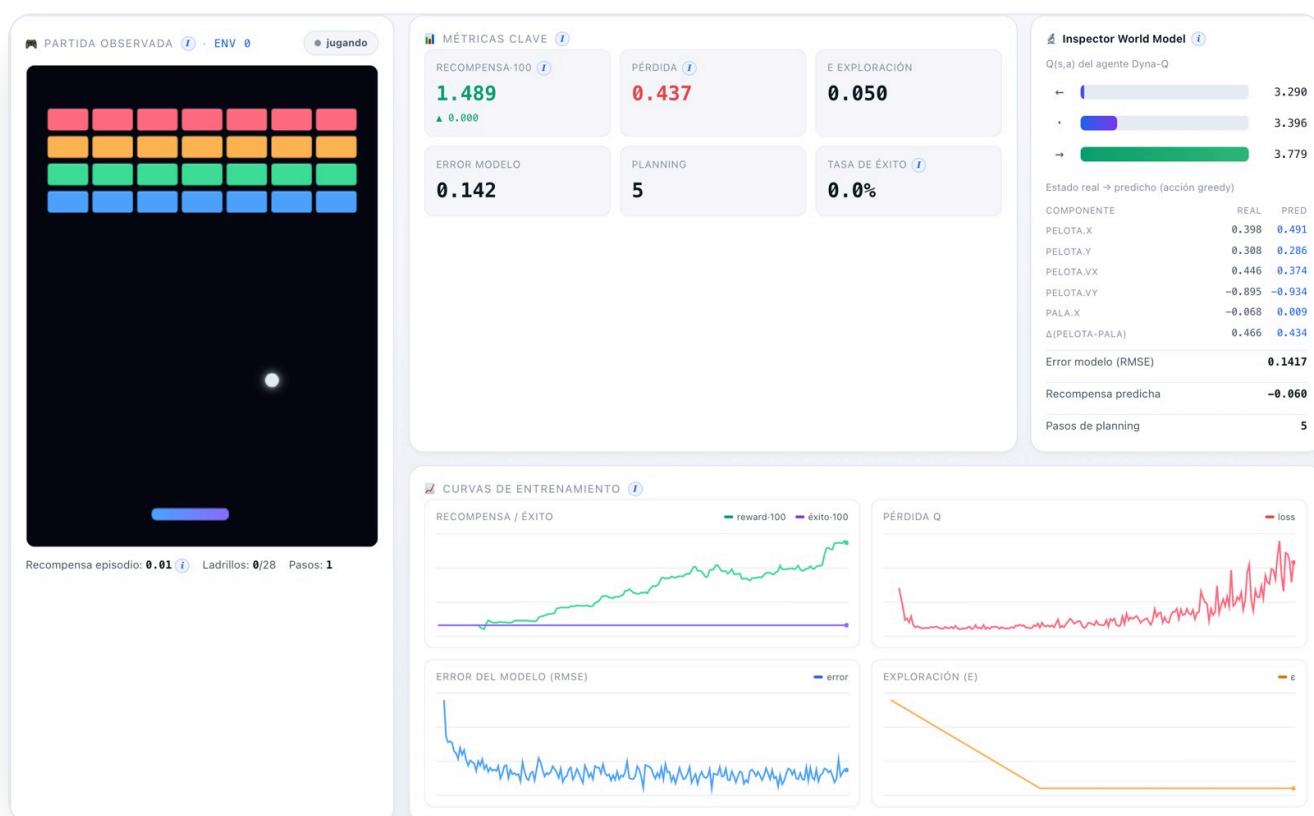
La decisión de diseño clave está en **qué predice el modelo**:

| Opción                                      | Qué predice                                       | Por qué la elegimos o no                                                  |
|---------------------------------------------|---------------------------------------------------|---------------------------------------------------------------------------|
| <b>Estado siguiente (s')</b>                | La posición absoluta de todo.                     | Más difícil: cualquier desvío en cada número se nota mucho. Descartada.   |
| <b>Incremento (<math>\Delta s</math>) ✓</b> | Solo <b>cuánto cambia</b> : $s' = s + \Delta s$ . | Más fácil y estable: los cambios por paso son pequeños y suaves. Elegida. |

## El experimento: objetivo, entorno y parámetros

| AJUSTE                  | VALOR   | QUÉ ES Y QUÉ EFECTO TIENE                                                    |
|-------------------------|---------|------------------------------------------------------------------------------|
| <b>capas del modelo</b> | 200·200 | El modelo de dinámica es algo más grande: tiene que aprender la física.      |
| <b>pasos_planning</b>   | 5       | Actualizaciones del Q-net con datos imaginados por cada paso real.           |
| <b>arranque_modelo</b>  | 1.000   | Pasos reales antes de empezar a entrenar el modelo (necesita datos primero). |
| <b>lr_modelo</b>        | 1e-3    | Ritmo de aprendizaje del modelo de dinámica (supervisado).                   |

## La interfaz



World Model entrenando: las métricas se adaptan (aparecen "Error modelo" y "Planning"). El inspector compara el estado real con el predicho por el modelo.

## Inspector World Model

Q(s,a) del agente Dyna-Q



Estado real → predicho (acción greedy)

| COMPONENTE                   | REAL   | PRED   |
|------------------------------|--------|--------|
| PELOTA.X                     | 0.398  | 0.491  |
| PELOTA.Y                     | 0.308  | 0.286  |
| PELOTA.VX                    | 0.446  | 0.374  |
| PELOTA.VY                    | -0.895 | -0.934 |
| PALA.X                       | -0.068 | 0.009  |
| $\Delta(\text{PELOTA-PALA})$ | 0.466  | 0.434  |

---

|                     |        |
|---------------------|--------|
| Error modelo (RMSE) | 0.1417 |
|---------------------|--------|

---

|                     |        |
|---------------------|--------|
| Recompensa predicha | -0.060 |
|---------------------|--------|

---

|                   |   |
|-------------------|---|
| Pasos de planning | 5 |
|-------------------|---|

Inspector del World Model: Q-values del Dyna-Q y la tabla real → predicho componente a componente. Aquí el error RMSE del modelo es 0.11.

## Resultados

El modelo aprende a predecir la física con un error **RMSE ~0.066** sobre estados normalizados (muy bajo: las predicciones siguen de cerca a la realidad, como se ve en el inspector). La recompensa subió de ~0.0 a **~0.71** en el navegador. Sin fugas: ~46 tensores constantes.



Curvas reales del World Model. La gráfica de "datos" muestra el error del modelo (RMSE) bajando: el modelo mejora su predicción del juego.

### ⚠ El problema

El **model bias**: si el modelo se equivoca, las imaginaciones largas acumulan error y el agente aprende de un mundo falso.

### ✓ La mejora encontrada

Predcir el **incremento  $\Delta s$**  (más estable) e imaginar a **corto plazo** (1 paso, reiniciando trayectorias terminadas). Resultado: modelo fiable (RMSE bajo) y política que mejora con datos imaginados.

## Interpretación: qué pasó al entrenar

### 📊 Qué observamos

- El modelo predice la física con un error **RMSE ~0.066** (muy bajo).
- La recompensa sube a ~0.71: por debajo de DQN, PPO y SAC.
- Memoria plana (~46 tensores) pese a tener dos redes que entrenar.

### ✓ Qué funcionó

- Predcir  **$\Delta s$**  e imaginar a **1 paso**: el modelo se mantiene honesto.
- Los datos imaginados **aceleran** el aprendizaje del Q-net.
- Es la mayor **eficiencia de muestra** de los cuatro.

### ↗ Qué mejoraríamos

- El **model bias** pone techo: imaginar de más enseña un mundo falso.
- Probar un **conjunto de modelos** y medir su incertidumbre para imaginar más sin riesgo.
- Alargar el horizonte imaginado solo cuando el modelo sea muy fiable.

## Comparación con el modelo del que parte (DQN)

El World Model es, en el fondo, **un DQN con un simulador encima**: decide con un Q-net idéntico; lo que cambia es **de dónde saca la experiencia**.

| Eje                 | DQN (01)               | World Model (04)                             |
|---------------------|------------------------|----------------------------------------------|
| Tipo                | Model-free.            | <b>Model-based</b> (Dyna-Q).                 |
| De dónde aprende    | Solo experiencia real. | Real + <b>imaginada</b> (5x por paso).       |
| Eficiencia en datos | Alta.                  | <b>La más alta...</b> si el modelo acierta.  |
| Su riesgo propio    | Sobreestimar Q.        | <b>Model bias</b> : imaginar un mundo falso. |
| Resultado aquí      | ~0.9 (corto).          | ~0.71: frenado por el model bias.            |

La promesa del World Model es **aprender con menos partidas reales**; el peaje es que su simulador tiene que ser bueno. Aquí lo es (RMSE bajo), pero el techo de recompensa recuerda que **imaginar nunca es del todo gratis**.

## Conclusiones

### En una frase

El World Model es **model-based**: aprende un **modelo de dinámica** y lo usa para **imaginar** experiencia (Dyna-Q), ganando eficiencia de muestra. El precio a vigilar es el **model bias**.

# Los cuatro, cara a cara

Misma tarea, mismo entorno, cuatro filosofías distintas.

| Algoritmo          | Familia                    | Política   | Qué aprende       | Exploración                | Señal propia         | Reward real*                         |
|--------------------|----------------------------|------------|-------------------|----------------------------|----------------------|--------------------------------------|
| <b>DQN</b>         | Model-free · valor         | off-policy | $Q(s,a)$          | $\epsilon$ -greedy         | TD-error, $\epsilon$ | $0 \rightarrow 0.9$ (>2 largo)       |
| <b>PPO</b>         | Model-free · actor-crítico | on-policy  | $\pi(a s) + V(s)$ | entropía                   | entropía, ventaja    | $0.1 \rightarrow 0.99$ (4.08 en CPU) |
| <b>SAC</b>         | Model-free · actor-crítico | off-policy | $\pi + 2 \cdot Q$ | máx. entropía ( $\alpha$ ) | temperatura $\alpha$ | $0 \rightarrow 0.99$                 |
| <b>World Model</b> | Model-based                | off-policy | dinámica + $Q$    | $\epsilon$ -greedy         | error del modelo     | $0 \rightarrow 0.71$                 |

\* Recompensa media de los últimos 100 episodios, medida en ejecuciones reales (navegador con WebGPU; el dato de PPO en CPU es de una corrida más larga). El throughput observado fue de ~24.000 exp/s en WebGPU frente a ~5.600 exp/s en CPU.

## La misma maquinaria, dimensión a dimensión

Y aquí están los cuatro enfrentados en cada decisión de diseño que hemos ido viendo capítulo a capítulo:

| Dimensión                  | DQN                        | PPO                      | SAC                                    | World Model                 |
|----------------------------|----------------------------|--------------------------|----------------------------------------|-----------------------------|
| <b>Familia</b>             | Valor                      | Actor-crítico            | Actor-crítico                          | Model-based                 |
| <b>Qué aprende</b>         | Valor de cada acción       | Estrategia + valor       | Estrategia + 2 valores                 | Física + valor              |
| <b>Nº de redes</b>         | 2                          | 2                        | 5                                      | 2                           |
| <b>Función de pérdida</b>  | Huber (TD)                 | Recortada + V + entropía | 3 pérdidas (min-Q, actor, $\alpha$ )   | Regresión + Huber           |
| <b>Memoria</b>             | Replay buffer              | Rollout (usa y tira)     | Replay buffer                          | Replay buffer               |
| <b>Exploración</b>         | $\epsilon$ -greedy         | Entropía fija            | Entropía con $\alpha$ <b>auto</b>      | $\epsilon$ -greedy          |
| <b>Eficiencia en datos</b> | Alta                       | Media                    | Alta                                   | <b>La más alta*</b>         |
| <b>Estabilidad</b>         | Media-alta                 | <b>La mejor</b>          | Alta                                   | Según el modelo             |
| <b>Coste por paso</b>      | Bajo                       | Bajo                     | Alto                                   | Medio                       |
| <b>Su riesgo propio</b>    | Sobreestimar Q             | Pasos grandes            | Cómputo                                | Model bias                  |
| <b>Brilla cuando...</b>    | Quieres un cimiento simple | Quieres cero sustos      | Los datos son caros y hay que explorar | Cada dato real cuesta mucho |

\* "La más alta" siempre que el modelo aprendido sea fiel; si no, el model bias lo penaliza.

### Qué llevarse

No hay un "mejor" universal: **DQN** es el cimiento más simple; **PPO** es el más estable y de uso general; **SAC** es el más eficiente en datos y explora solo; el **World Model** es el más ambicioso (aprende el juego) y el más delicado. Entender los cuatro es entender el mapa del RL profundo.

# Glosario

Todos los términos de la guía, en una página, para consultar de un vistazo.

**Agente** — quien aprende y decide; aquí, la(s) red(es) neuronal(es).

**Entorno** — el mundo con el que interactúa el agente; aquí, el juego de Arkanoid.

**Estado (s)** — lo que el agente observa en un instante (6 números).

**Acción (a)** — lo que puede hacer ( $\leftarrow \cdot \rightarrow$ , 3 acciones).

**Recompensa (r)** — premio/castigo numérico de cada paso.

**Retorno (G)** — suma de recompensas futuras, descontadas.

**Descuento ( $\gamma$ )** — cuánto pesa el futuro (0.99 = previsor).

**Política ( $\pi$ )** — la estrategia: de un estado a una acción (o probabilidades).

**Valor  $V(s)$**  — recompensa futura esperada desde un estado.

**Q-valor  $Q(s,a)$**  — recompensa futura esperada de una acción en un estado.

**Episodio** — una partida completa (hasta perder, ganar o agotar el tiempo).

**Exploración/explotación** — probar lo nuevo vs aprovechar lo conocido.

**$\epsilon$ -greedy** — explorar al azar con probabilidad  $\epsilon$  (que decae).

**Replay buffer** — memoria de experiencias para muestrear lotes.

**On-policy / off-policy** — aprender solo de la política actual / también de datos viejos.

**Red objetivo** — copia lenta de la red que da un blanco estable.

**TD-error** — diferencia entre lo predicho y el objetivo de Bellman; “sorpresa”.

**Ventaja  $A(s,a)$**  — cuánto mejor es una acción que la media:  $Q - V$ .

**GAE** — forma de estimar la ventaja con baja varianza.

**Entropía** — cuán aleatoria es la política (más entropía = más exploración).

**Temperatura  $\alpha$**  — peso de la entropía en SAC, ajustado solo.

**Model-free / model-based** — sin / con modelo del entorno.

**Bellman** — ecuación recursiva: valor = recompensa + valor del futuro.

**Gradiente / learning rate** — dirección y tamaño del ajuste de los pesos.

**MLP / ReLU** — red de capas densas / activación no lineal  $\max(0, z)$ .

**Shaping** — recompensa densa auxiliar que acelera sin cambiar el óptimo.

# Para seguir aprendiendo

Lecturas y experimentos para profundizar.

La progresión de este laboratorio — **valor** (DQN) → **política** (PPO) → **política + entropía** (SAC) → **modelo del mundo** (World Model) — es la misma escalera que recorre el campo del aprendizaje por refuerzo.

## Lecturas recomendadas

**Sutton & Barto — *Reinforcement Learning: An Introduction***. El libro de referencia, gratuito online. Empieza por los capítulos de MDP, Bellman y Q-learning: reconocerás todo lo de “Fundamentos”.

**OpenAI Spinning Up in Deep RL**. Tutorial práctico con implementaciones limpias de PPO, SAC y más.

**Mnih et al. (2015) — *Human-level control through deep RL***. El paper original de DQN (Atari): replay buffer + red objetivo.

**Schulman et al. (2017) — *Proximal Policy Optimization***. El objetivo recortado que viste en PPO.

**Haarnoja et al. (2018) — *Soft Actor-Critic***. Máxima entropía y temperatura automática (SAC).

**Sutton (1991) — *Dyna*** y **Ha & Schmidhuber (2018) — *World Models***. La idea de aprender un modelo e “imaginar”.

## Experimentos que puedes hacer con este laboratorio



### Toca $\gamma$ y $\epsilon$

Baja  $\gamma$  a 0.8: ¿se vuelve cortoplacista? Acelera el decaimiento de  $\epsilon$ : ¿explora menos y se estanca?



### Cambia la red

Prueba capas más grandes o más pequeñas en constantes `.js` y mira el efecto en la curva.



### Rediseña las recompensas

Sube el castigo por perder o el premio por romper: observa cómo cambia el comportamiento del agente.



### Compara algoritmos

Entrena los cuatro el mismo tiempo y compara recompensa, estabilidad y velocidad.

PRUÉBALO

# Descárgalo y entrénalo tú

Todo corre en local; el entrenamiento, en tu navegador con WebGPU.

## Repositorio

[github.com/rcanalescoder/DRLArkanoid](https://github.com/rcanalescoder/DRLArkanoid)

## 1 · Clona e instala

terminal

INSTALACIÓN

```
git clone https://github.com/rcanalescoder/DRLArkanoid.git && cd DRLArkanoid
npm install           # vite + @tensorflow/tfjs + backend WebGPU
```

## 2 · Arranca el laboratorio

terminal

ARRANCAR

```
npm run dev           # abre http://localhost:5173 y pulsa ► Entrenar
./arrancar.sh         # alternativa: arranca y, si ya estaba, rearranca
```

## 3 · Verifica o entrena por consola (sin navegador)

terminal

VERIFICACIÓN

```
npm run verificar     # entrena los 4 algoritmos (CPU) y reporta si aprenden
node scripts/entrenar.mjs dqn --pasos 40000 --envs 128 # entrena uno y muestra trazas
```

Requiere Node  $\geq 18$  y un navegador con WebGPU (Chrome/Edge/Safari recientes). Si no hay WebGPU, el laboratorio cae automáticamente a WebGL y, en último caso, a CPU.

# Cómo está organizado el código

Dónde vive cada pieza del laboratorio.

```
src/
├── nucleo/                # infraestructura común
│   ├── constantes.js      # entorno, recompensas, hiperparámetros
│   ├── busEventos.js      # pub/sub: desacopla motor y UI
│   ├── gestorTensores.js  # tf.tidy, soft update, paso de gradiente
│   └── trazas.js          # logging estructurado de métricas
├── entorno/
│   ├── entornoArkanoid.js # el juego (física de paso fijo)
│   └── gestorEntornos.js  # pools headless + visual
├── agentes/              # ←* AQUÍ VIVEN LAS REDES Y LOS ALGORITMOS
│   ├── agenteDQN.js       # Double DQN
│   ├── agentePPO.js       # actor-crítico + GAE + clip
│   ├── agenteSAC.js       # 2 críticos +  $\alpha$  automática
│   ├── agenteWorldModel.js # modelo de dinámica + Dyna-Q
│   └── redes/constructorRedes.js # fábrica de MLPs
├── datos/
│   ├── replayBuffer.js    # off-policy (DQN, SAC, WM)
│   ├── replayPrioritario.js # PER con sum-tree
│   └── rolloutBuffer.js   # on-policy (PPO) + GAE
├── entrenamiento/        # ←* EL BUCLE DE ENTRENAMIENTO
│   ├── orquestador.js     # el ciclo, sin DOM
│   └── metricas.js        # recolector de métricas reales
├── ui/                   # interfaz (paneles, curvas, inspectores)
└── main.js               # arranque + detección de backend
```

**Las redes**                    agentes/\*.js + agentes/redes/constructorRedes.js

**El entrenamiento**            entrenamiento/orquestador.js (el ciclo) y el método entrenar() de cada agente (la regla de actualización).

**El juego**                    entorno/entornoArkanoid.js (física) y gestorEntornos.js (los dos pools).

**La memoria**                datos/replayBuffer.js (off-policy) y datos/rolloutBuffer.js (on-policy).

# Algoritmo a algoritmo

Fragmentos reales: la red, el entrenamiento y un extra de cada uno.

## La fábrica de redes (común a todos)

src/agentes/redes/constructorRedes.js · crearMLP()

LA RED

```
const modelo = tf.sequential({ name: nombre });
capasOcultas.forEach((unidades, i) => {
  modelo.add(tf.layers.dense({
    units: unidades, activation: "relu",
    inputShape: i === 0 ? [dimEntrada] : undefined,
    kernelInitializer: "heNormal", // buena inicialización para ReLU
  }));
});
modelo.add(tf.layers.dense({ units: dimSalida, activation: activacionSalida }));
```

Un MLP parametrizable. DQN/WM lo usan con 3 salidas (Q); PPO/SAC para el actor (3 logits) y la crítica (1 valor).

## PPO · el objetivo recortado

src/agentes/agentePPO.js · entrenar() (minibatch)

EL ENTRENAMIENTO

```
const ratio = logpA.sub(logpViejoMb).exp(); //  $\rho = \pi_{nueva} / \pi_{vieja}$ 
const surr1 = ratio.mul(advMb);
const surr2 = ratio.clipByValue(1 - eps, 1 + eps).mul(advMb); // recorte
const lossPol = surr1.minimum(surr2).mean().mul(-1); // objetivo recortado
const entropia = probs.mul(logp).sum(1).mean().mul(-1);
const lossVal = valor.sub(retMb).square().mean(); // crítica: MSE contra el retorno
return lossPol.add(lossVal.mul(coefValor)).sub(entropia.mul(coefEntropia));
```

El corazón de PPO: tomar el mínimo entre el objetivo normal y el recortado evita pasos demasiado grandes.

## SAC · la temperatura $\alpha$ que se aprende sola

src/agentes/agenteSAC.js · entrenar() (paso de  $\alpha$ )

UN EXTRA

```
//  $\alpha$  se ajusta para acercar la entropía de la política a la entropía objetivo.
const lossAlphaT = pasoGradiente(this.optAlpha, () => {
  const term = this.logAlpha.mul(-1).mul(logpC.add(this.targetEntropy));
  return probsC.mul(term).sum(1).mean();
}, [this.logAlpha]);
```

$\alpha$  no es un hiperparámetro fijo: es una variable entrenable. Esto le quita al usuario el trabajo de afinarla.

## World Model · entrenar el modelo de dinámica

src/agentes/agenteWorldModel.js · \_entrenarModelo()

UN EXTRA

```
const lossT = pasoGradiente(this.optModelo, () => {
  const out = this.modelo.predict(entrada);           // entrada = [s @ one-hot(a)]
  const dsPred = out.slice([0, 0], [B, this.dimEstado]); //  $\Delta s$  predicho
  const rPred = out.slice([0, this.dimEstado], [B, 1]); // r predicha
  const donePred = out.slice([0, this.dimEstado + 1], [B, 1]);
  return tf.losses.meanSquaredError(dsObjetivo, dsPred) // aprender la física...
    .add(tf.losses.meanSquaredError(rT, rPred))         // ...la recompensa...
    .add(tf.losses.sigmoidCrossEntropy(doneT, donePred).mul(0.5)); // ...y el fin de episodio
}, this._varsModelo);
```

El modelo se entrena de forma **supervisada**: predecir ( $\Delta s$ , r, done) a partir de (s, a). Con él, el agente "imagina".

# Licencia MIT

Úsalo, modifícalo y compártelo libremente.

|                    |                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------|
| <b>Autor</b>       | Roberto Canales Mora · con Claude Chat / Code                                                   |
| <b>Licencia</b>    | MIT                                                                                             |
| <b>Repositorio</b> | <a href="https://github.com/rcanalescoder/DRLArkanoid">github.com/rcanalescoder/DRLArkanoid</a> |
| <b>Entorno</b>     | Vite + TensorFlow.js (WebGPU / WebGL / CPU). Probado en macOS (Apple Silicon M4).               |

## LICENSE

## TEXTO COMPLETO

Licencia MIT

Copyright (c) 2026 Roberto Canales Mora

Por la presente se concede permiso, sin cargo, a cualquier persona que obtenga una copia de este software y de los archivos de documentación asociados (el "Software"), para utilizar el Software sin restricción, incluyendo sin limitación los derechos a usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del Software, y a permitir a las personas a las que se les proporcione el Software a hacer lo mismo, sujeto a las siguientes condiciones:

El aviso de copyright anterior y este aviso de permiso se incluirán en todas las copias o partes sustanciales del Software.

EL SOFTWARE SE PROPORCIONA "TAL CUAL", SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA, INCLUYENDO PERO NO LIMITADO A GARANTÍAS DE COMERCIALIZACIÓN, IDONEIDAD PARA UN PROPÓSITO PARTICULAR Y NO INFRACCIÓN. EN NINGÚN CASO LOS AUTORES O PROPIETARIOS DE LOS DERECHOS DE AUTOR SERÁN RESPONSABLES DE NINGUNA RECLAMACIÓN, DAÑOS U OTRAS RESPONSABILIDADES, YA SEA EN UNA ACCIÓN DE CONTRATO, AGRAVIO O DE OTRO MODO, QUE SURJA DE, FUERA DE O EN CONEXIÓN CON EL SOFTWARE O EL USO U OTROS NEGOCIOS EN EL SOFTWARE.